Check for updates

ACM DIGITAL LIBRARY | Association for Computing Machinery | acm open

Latest updates: https://dl.acm.org/doi/10.1145/3735968

PDF Download
3735968.pdf
08 April 2026
Total Citations: 8
Total Downloads: 3894

Published: 11 July 2025
Online AM: 15 May 2025
Accepted: 08 May 2025
Revised: 01 May 2025
Received: 14 June 2023

Citation in BibTeX format

SURVEY

Detecting Misuse of Security APIs: A Systematic Review

ZAHRA MOUSAVI, The University of Adelaide, Adelaide, SA, Australia

CHADNI ISLAM, Edith Cowan University, Perth, WA, Australia

MUHAMMAD ALI BABAR, The University of Adelaide, Adelaide, SA, Australia

ALSHARIF ABUADBBA, Commonwealth Scientific and Industrial Research Organisation, Canberra, ACT, Australia

KRISTEN MOORE, Commonwealth Scientific and Industrial Research Organisation, Canberra, ACT, Australia

Open Access Support provided by:

Commonwealth Scientific and Industrial Research Organisation

The University of Adelaide

Edith Cowan University

Detecting Misuse of Security APIs: A Systematic Review

ZAHRA MOUSAVI, Centre for Research on Engineering Software Technologies (CREST) & Adelaide University, Cyber Security CRC, CSIRO's Data61, Adelaide, Australia
CHADNI ISLAM, Edith Cowan University, Joondalup, Australia
MUHAMMAD ALI BABAR, Centre for Research on Engineering Software Technologies (CREST) & Adelaide University, Adelaide, Australia
ALSHARIF ABUADBBA, CSIRO's Data61, Sydney, Australia
KRISTEN MOORE, CSIRO's Data61, Melbourne, Australia

Security Application Programming Interfaces (APIs) are crucial for ensuring software security. However, their misuse introduces vulnerabilities, potentially leading to severe data breaches and substantial financial loss. Complex API design, inadequate documentation, and insufficient security training often lead to unintentional misuse by developers. The software security community has devised and evaluated several approaches to detecting security API misuse to help developers and organizations. This study rigorously reviews the literature on detecting misuse of security APIs to gain a comprehensive understanding of this critical domain. Our goal is to identify and analyze security API misuses, the detection approaches developed, and the evaluation methodologies employed along with the open research avenues to advance the state-of-the-art in this area. Employing the systematic literature review (SLR) methodology, we analyzed 69 research articles. Our review has yielded (a) identification of 6 security API types; (b) classification of 30 distinct misuses; (c) categorization of detection techniques into heuristic-based and ML-based approaches; and (d) identification of 10 performance measures and 9 evaluation benchmarks. The review reveals a lack of coverage of detection approaches in several areas. We recommend that future efforts focus on aligning security API development with developers' needs and advancing standardized evaluation methods for detection technologies.

CCS Concepts: • **Security and privacy → Software security engineering;**

Additional Key Words and Phrases: Security API, secure software development, API misuse, misuse detection

ACM Reference Format:
Zahra Mousavi, Chadni Islam, Muhammad Ali Babar, Alsharif Abuadbba, and Kristen Moore. 2025. Detecting Misuse of Security APIs: A Systematic Review. *ACM Comput. Surv.* 57, 12, Article 303 (July 2025), 39 pages. https://doi.org/10.1145/3735968

The work has been supported by the Cyber Security Research Centre Limited, whose activities are partially funded by the Australian Government's Cooperative Research Centres Programme.
Authors' Contact Information: Zahra Mousavi, Centre for Research on Engineering Software Technologies (CREST) & Adelaide University, Cyber Security CRC, CSIRO's Data61, Adelaide, South Australia, Australia; e-mail: seyedehzahra.mosavi@adelaide.edu.au; Chadni Islam, Edith Cowan University, Joondalup, Western Australia, Australia; e-mail: c.islam@ecu.edu.au; Muhammad Ali Babar, Centre for Research on Engineering Software Technologies (CREST) & Adelaide University, Adelaide, South Australia, Australia; e-mail: ali.babar@adelaide.edu.au; Alsharif Abuadbba, CSIRO's Data61, Sydney, New South Wales, Australia; e-mail: sharif.abuadbba@data61.csiro.au; Kristen Moore, CSIRO's Data61, Melbourne, Victoria, Australia; e-mail: kristen.moore@data61.csiro.au.



This work is licensed under a Creative Commons Attribution 4.0 International License.

© 2025 Copyright held by the owner/author(s).
ACM 0360-0300/2025/07-ART303
https://doi.org/10.1145/3735968

1 Introduction

Security **Application Programming Interfaces** (**APIs**) are integral to modern software development that serves billions of users through web or mobile apps. They offer developers specific functionalities, such as data encryption or access control, to address security concerns. Cryptography and SSL/TLS APIs, for example, are widely used to ensure data confidentiality and secure communications [1]. Although security APIs are highly beneficial for ensuring software security, their incorrect use, known as misuse, inadvertently leads to software vulnerabilities, posing a significant risk to overall system security and potentially exposing millions of users to sensitive data breaches and financial losses [1–6].

Figure 1 illustrates an SSL/TLS API misuse, where a developer uses the API to establish a secure connection with a server (Step ①) but chooses to *trust all server hostnames* (Step ②). By exploiting the misuse, a malicious actor can impersonate a valid server, intercept the communication between a user and the application, and obtain unauthorized access to the user's personal information (Step ③). Many developers do not fully understand the implications of *trusting all hostnames* while using an SSL/TLS API [1]. The complex design of security APIs, poor documentation, and inadequate training significantly hamper developers' understanding of security APIs and contribute to the widespread misuse of these APIs [7]. A comprehensive study of ten thousand Android applications by Krüger et al. [6] revealed that nearly 95% of the applications contain at least one cryptography API misuse. Similarly, an analysis of over two thousand open-source Java projects on GitHub found that 72% of them suffer from at least one cryptography API misuse [8].

Given the increasing realization of the potentially devastating consequences of misuse of security APIs, there has been significant interest in devising and evaluating effective and efficient approaches to detecting security API misuses. Nevertheless, the relevant literature is dispersed and lacks coherent analysis and synthesis. This gap underscores the need for a systematic survey that could assist both researchers and practitioners in gaining a comprehensive understanding of state-of-the-art approaches in this field. Therefore, we aim at conducting a thorough and holistic analysis of the literature specifically concerning the detection of security API misuse. We leverage the **Systematic Literature Review** (**SLR**) methodology and focus on four key dimensions: *the security APIs studied, the types of misuses reported, the detection techniques proposed*, and *the evaluation methods employed* that require attention. This leads to the formation of four primary research questions: **(RQ1)** *What **security APIs** are commonly analyzed in the context of misuse detection across software applications?* **(RQ2)** *What security API **misuses** are commonly investigated in the literature for detection across software applications?* **(RQ3)** *What **techniques** are commonly employed to detect security API misuses?* **(RQ4)** *What **evaluation strategies** are commonly utilized to assess the performance of misuse detection techniques?* Our SLR, covering 69 peer-reviewed studies, yields the following main **contributions**:

- A large-scale survey of the literature on misuse detection for security APIs using a systematic review method.
- An in-depth discussion on the security APIs studied from misuse perspective and their misuses.
- A taxonomic analysis of the existing approaches to detecting misuse of security APIs.
- A critical rundown of the strategies, benchmarks, and metrics used for evaluating the proposed approaches.
- A set of open issues that can form the future research agenda for enhancing and evaluating security API use.

The remainder of this article is structured as follows: Section 2 provides an overview of security APIs and their misuses. Section 3 describes the scientific methodology of this SLR. Our findings of

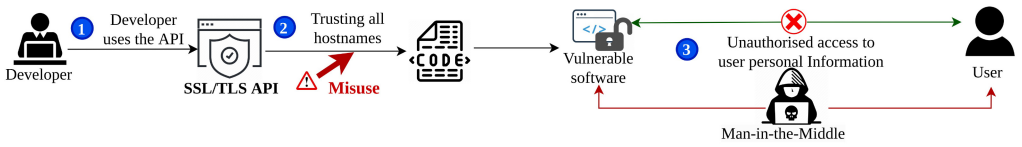


Fig. 1. A misuse of SSL/TLS API leading to the leakage of user personal information.

security APIs and misuses are presented in Sections 4 and 5, respectively. Section 6 analyzes the misuse detection techniques, while Section 7 presents the evaluation strategies. Section 8 discusses open issues and future research directions. Section 9 addresses the threats to the validity of our findings. Finally, the article concludes in Section 10.

2 Preliminaries

This section presents an overview of security APIs and potential misuses that developers may make while using them.

2.1 Security API

An API is essentially a set of programming instructions that allows software components to interact with each other, making it easier for developers to build complex software systems [9]. Security APIs are a subset of APIs that provide developers with security functionalities, such as confidentiality, data integrity, authentication, and authorization [10]. Confidentiality is the process of protecting sensitive information from unauthorized disclosure [11]. Data encryption, provided by cryptography APIs, is a primary means to ensure the confidentiality of sensitive data. Data integrity mechanisms ensure the accuracy, trustworthiness, and validity of data by protecting it from unauthorized changes throughout its life cycle. Cryptography and SSL/TLS APIs are among the popular security APIs that provide security functionalities to ensure data integrity [6]. Authentication is the process of verifying the identity of legitimate users or systems before granting data access [11], while authorization specifies access rights and privileges to resources [11]. OAuth APIs [12] are widely used for both authentication and authorization purposes, enabling users to grant access to their resources and data to third-party applications without exposing their credentials.

2.2 Misuse of Security APIs

APIs function correctly only when specific constraints on their inputs, outputs, and invocation context are met, as outlined in the API specification. Any deviation from these specifications, known as misuse, can lead to various problems such as performance degradation, compatibility issues, and unexpected behavior. However, misusing security APIs can have far more severe consequences, including exposing confidential data and putting entire systems at risk. Moreover, misusing security APIs can lead to non-compliance with data protection regulations such as GDPR [13] and HIPAA [14], potentially resulting in hefty fines and severe damage to a company's reputation. To mitigate these risks, developers must strictly adhere to the latest specifications when using security APIs. For instance, secure communication via an SSL/TLS API requires a client to verify the server's hostname. Figure 1 illustrates a violation of this specification, demonstrating how neglecting hostname verification in the SSL/TLS API can lead to a Man-in-the-Middle attack. In this scenario, attackers can impersonate a legitimate server, intercepting communication between the user and the application, thereby gaining unauthorized access to the user's personal information.

As digital transformation accelerates, the use of APIs, including security APIs, is growing. The increasing interconnectivity of systems and the proliferation of IoT devices expand the attack surface, making the correct use of security APIs more critical than ever. However, the research reveals

Table 1. Categories of Key Terms used for Defining Search String

Category	Synonyms and Relevant Terms
Security	*secur* OR crypto**
API	*api OR librar* OR interface*
Misuse	*misuse OR incorrect OR insecure OR vulnerability**
Detection	*detect* OR "static analysis" OR "dynamic analysis" OR "program analysis" OR "code analysis"*

a concerning trend of widespread misuse of these APIs among developers. [6, 8], which can be attributed to several factors. Firstly, the complex and nuanced operation of security APIs coupled with the inherent complexity of API design can hinder developers' understanding and correct implementation [7]. Security API use is further complicated by poorly written documentation, often lacking clear usage examples, and sometimes including insecure code examples. For instance, misuses were identified in code samples within the documentation of an OAuth API, which can mislead developers who copy them without understanding security implications [15]. Additionally, developers often use unreliable forum posts for guidance, which can propagate instances of misuse [16]. Lastly, the lack of cybersecurity training, coupled with the evolving threat landscape surrounding security APIs, makes it challenging for developers to stay current on the latest best practices for security APIs [17].

3 Research Methodology

We conducted a SLR to gain insight into misuse detection approaches for security APIs. SLR is broadly adopted as a research methodology in Evidence-Based Software Engineering [18] as it provides a reliable, rigorous, and auditable technique for assessing and interpreting a research topic [19]. We followed the SLR guideline provided by Kitchenham et al. [19]. The steps of our review protocol are elaborated in Sections 3.1–3.4.

3.1 Search Strategy

We followed the guidelines provided by Kitchenham et al. [19] to develop our search strategy, ensuring that we obtain the highest number of relevant studies. The search strategy is comprised of the following steps.

3.1.1 Search Method. We applied an automated database search method [19] to digital search engines and databases to obtain relevant studies. We used Scopus **Digital Library** (**DL**) as the primary source, which is the largest academic literature database, indexing over 5,000 publishers worldwide, including relevant sources like Elsevier and Springer [20, 21]. To complement Scopus results, we also used the two prominent academic DLs – IEEE Xplore and ACM DL [22].

3.1.2 Search String. We crafted a comprehensive search string following the guidelines presented by Kitchenham et al. [19]. We initiated our search using four main keywords: "security", "API", "misuse", and "detection". To broaden our search, we also considered synonyms for these terms. We reviewed titles, abstracts, and keywords from some relevant articles to ensure we captured associated synonyms. Synonyms that returned an excess of irrelevant results, such as "flaw", were excluded from our search string. We conducted a series of pilot searches to ensure the inclusion of relevant articles that we were already aware of. Ultimately, we organized the keywords and their pertinent synonyms into four categories, which are shown in Table 1. We used the union (AND) of the categories to conduct searches in titles, abstracts, and keywords of articles on Scopus, IEEE Xplorer, and ACM DL.

Table 2. Inclusion and Exclusion Criteria

Inclusion Criteria
I1: Articles that address misuse detection for security APIs, including articles that use either an automated or manual approach or rely on existing tools to verify the usage of security APIs.
I2: Articles published in peer-reviewed venues that are ranked by CORE or Scimago.
I3: Articles written in English and their full text are accessible.
Exclusion Criteria
E1: Articles that target detecting vulnerabilities in the internal design or implementation of security APIs, not misuses.
E2: Articles that target detecting misuses of generic APIs.
E3: Short articles less than six pages.
E4: Book chapters, dissertations, and non-peer-reviewed publications (e.g., keynotes, editorials, tutorials, and panel discussions).

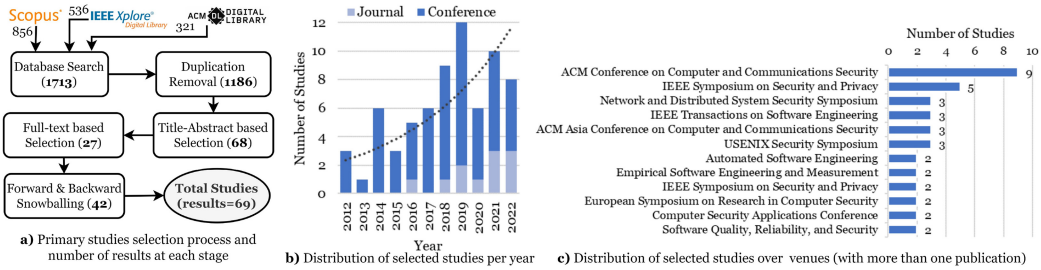


Fig. 2. Primary studies selection process and their distribution over years and publication venues.

3.2 Inclusion-exclusion Criteria

We applied the inclusion and exclusion criteria outlined in Table 2 to filter the retrieved studies. We included studies on detecting misuse of security APIs in line with our research objectives and RQs (I1). The criteria were iteratively refined during the selection process to ensure an accurate selection of relevant articles. For instance, we excluded studies that focused on vulnerabilities in the internal design or implementation of security APIs (E1) or targeted at detecting misuses of generic APIs (E2). We also introduced E3 and E4 to exclude publications, such as short articles, that lack sufficient information to fully address the RQs. Moreover, we adopted a venue assessment criterion (I2), similar to methods used in previous studies [21, 23, 24], to include only high-quality articles. This criterion involved including only articles published in ranked conferences according to the CORE ranking,[1] and ranked journals according to the Scimago database.[2] Both databases employ meticulous and comprehensive evaluation methodologies, considering various factors to assess the quality and impact of venues. By utilizing these databases, we were able to identify high-quality articles effectively.

3.3 Selection of the Primary Studies

Figure 2(a) illustrates the different phases involved in the study selection process. In November 2022, we executed the search string in our data sources without any time limit on publication year, resulting in 1,713 studies. To remove duplicates, we initially compiled a pool of unique articles from Scopus, totaling 856. Subsequently, we supplemented this pool with additional unique articles from IEEE Xplore (271 articles) and then ACM DL (59 articles). Next, we refined our search by applying the inclusion-exclusion criteria outlined in Table 2. This involved initial screening based on title and abstract, resulting in the inclusion of 59 articles from Scopus, 8 from IEEE Xplore, and 1 from ACM DL. Subsequently, further refinement was performed through full-text assessment, leading to

[1] http://portal.core.edu.au/conf-ranks
[2] https://www.scimagojr.com/journalrank.php

the selection of 26 articles from Scopus and 1 from IEEE Xplore. Furthermore, we used forward and backward snowballing techniques [25] to ensure the maximum number of relevant articles were included in our review. This involves examining the citations and references of the selected articles to identify any missing relevant articles. Finally, 69 articles were included in our SLR, and their details can be found in our online appendix [26]. Each article in the review is assigned a unique identifier (S#). At each stage of the selection process, we meticulously deliberated and addressed any uncertainties through discussions among all the authors to minimize the risk of selection bias.

3.4 Data Extraction and Synthesis

We developed a **Data Extraction Form** (**DEF**) that comprises 14 data items essential for addressing our RQs, which we elaborated in our online appendix [26]. Data items (D1-D6) include demographic information such as title, author, venue, publication year, and publisher. To simplify the analysis of the extracted data relevant to our RQs, we categorized the data items into the following groups: RQ1 (D7-D8: security APIs, language), RQ2 (D9-D10: misuses, consequences), RQ3 (D11-D14: technique, modeling input data type, testing input data type, output type), and RQ4 (D15-D18: evaluation strategy, evaluation metrics, dataset, misuses reported). A pilot study was conducted on 12 articles to refine the DEF for capturing the necessary information in the most effective and summarized form.

Data Synthesis: We used descriptive statistics to analyze demographic information data items, while thematic analysis was used to analyze RQ-relevant data items. To conduct thematic analysis, we followed the steps outlined in the guideline by [27]. Firstly, we familiarized ourselves with the data by reading and examining the extracted data. Next, we generated initial codes to capture security APIs, misuses, detection techniques, and evaluation methods. Then, we searched for themes and generated potential themes for each data item by merging the corresponding initial codes based on their similarities. We reviewed the themes and mapped them iteratively to ensure all codes and themes were accurately allocated. Finally, we reviewed the synthesized results for each RQ and resolved any disagreements through regular meetings to finalize the answers to RQs.

3.5 Distribution of Studies

Figure 2(b) depicts the distribution of the 69 primary studies included in this review across different years. No relevant studies were identified before 2012, but since then, there has been an upward trend in publications toward 2022. This increasing trend signifies a growing interest from the research community in security API misuse detection, highlighting the need for a systematic analysis of this evolving field. The figure also shows the number of articles based on the venue type, including journals and conferences. Notably, a substantial majority–over 84%–of the selected articles (58 out of 69) were published in conferences. Figure 2(c) provides further insight into publication venues, including journals and conferences with more than one publication. Security-specific venues are well-represented, with the ACM Conference on **Computer and Communications Security** (**CCS**) and the IEEE Symposium on **Security and Privacy** (**S&P**) emerging as the most popular venues with 9 and 5 publications, respectively. Software engineering venues are also present (e.g., Automated Software Engineering with 2 articles), demonstrating the interdisciplinary nature of this field.

4 RQ1: Security APIs

This section presents our findings regarding RQ1, which focuses on the security APIs researchers have studied for the purpose of misuse detection (Section 4.1) and their misuse trend within real-world software (Section 4.2).

Table 3. Security APIs and their Mappings with Primary Studies (Number of Primary Studies Indicated in Parentheses)

APIs	Functionality	Language	Instances	Study Refs
Cryptographic Primitives (43)	Confidentiality Data Integrity Authentication	Java	Java Cryptography Architecture (JCA), Java Cryptography Extension (JCE), BouncyCastle (BC), Jasypt, Keyczar, GNU Crypto, SunJCE, SpoungyCastle, LP11	S1, S2, S3, S4, S5, S7, S10, S11, S12, S15, S16, S17, S18, S20, S23, S27, S29, S31, S32, S34, S35, S36, S40, S41, S42, S43, S44, S45, S46, S55, S58, S61, S63, S64, S65, S66
		Python	PyCrypto, PyNaCl, M2Crypto, cryptography.io, Keyczar, ucryptolib	S20, S59, S60
		C/C++	CommonCrypto, Libsodium, Nettle, TomCrypt, LibTomCrypt, Libgcrypt, WolfCrypt	S6, S13, S28
		JavaScript	WebCrypto APIs	S56
		Go	Go cryptographic APIs	S62
SSL/TLS (26)	Confidentiality Data Integrity Authentication	Java	Java Secured-Socket Extension (JSSE)	S1, S2, S3, S8, S14, S19, S24, S26, S27, S31, S33, S34, S39, S43, S44, S45, S46, S57, S58, S66, S67, S68
		C/C++	OpenSSL, GnuTLS, Libcrypto, Libcrypt, Cryptlib, WolfSSL	S9, S21, S22, S25, S26, S68
OAuth (9)	Authentication Authorization	-	OAuth APIs provided by service providers such as Google or Facebook	S30, S47, S48, S49, S50, S51, S52, S53, S54
Fingerprint (1)	Authentication Authorization	Java	Google Fingerprint API	S37
Spring Security (1)	Authentication Authorization	Java	Spring framework	S38
SafetyNet Attestation (1)	Device/App Integrity	Java	Google SafetyNet Attestation	S69

4.1 API Taxonomy

Our analysis revealed APIs from various contexts studied for misuse detection. Based on their primary purpose, we established a taxonomy of six API categories: ***Cryptographic primitives***, for essential low-level cryptographic functions, ***SSL/TLS***, for secure network communications, ***OAuth***, for access delegation without sharing credentials, ***Fingerprint***, for fingerprint-based authentication, ***Spring Security***, for integration of authentication and authorization mechanisms, and ***SafetyNet Attestation***, for device and application integrity checks. We further identified key security functionalities addressed by these APIs, including confidentiality, integrity (comprising data, device, and application integrity), authentication, and authorization. Table 3 presents API categories with further details on their functionalities, supported programming languages, specific API instances, and references from the reviewed studies. As shown, cryptographic primitives (31 studies) have received the most research attention, followed by SSL/TLS (14 studies). Notably, there are 12 additional studies that investigated both APIs. OAuth has been the focus of 9 studies, while Fingerprint, Spring Security, and SafetyNet Attestation each have been explored in one study.

Regarding the programming language, the majority of research appears to be focused on Java security APIs, as indicated by Table 3. This focus could stem from the complex design of Java APIs [S20, S60], highlighting the demand for the development of misuse detection approaches in Java. Additionally, Java holds significant popularity among developers across various domains and platforms such as web applications, mobile apps, enterprise systems, and embedded software [28]. Aside from Java, there were studies dedicated to security APIs in other programming languages, including C/C++ (8 studies), Python (3 studies), JavaScript (1 study), and Go (1 study) (Table 3).

The following sections provide an overview of each security API, focusing on key components and functionalities.

4.1.1 Cryptographic Primitives APIs. Cryptography is an essential component of secure software development as it plays a crucial role in maintaining confidentiality, data integrity, and

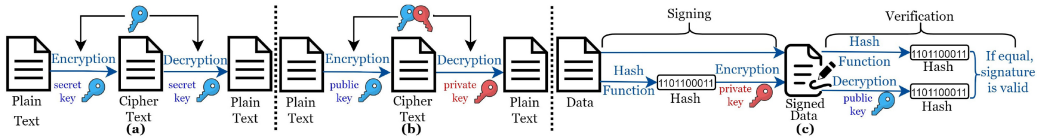


Fig. 3. (a) Symmetric encryption, (b) Asymmetric encryption, and (c) Signing and verification in digital signature.

authenticity. Developers often utilize APIs that implement cryptography primitives (referred to as crypto APIs, hereafter) to integrate these features into their software. Cryptography primitives are fundamental building blocks of cryptography. They consist of low-level functions including ***(i) symmetric encryption***, ***(ii) asymmetric encryption***, ***(iii) hash and message authentication code***, ***(iv) key derivation***, ***(v) key storage***, and ***(vi) pseudorandom number generator***.

Symmetric Encryption: Symmetric encryption algorithms or *ciphers*, secure data by converting *plaintext* into *ciphertext* that only authorized entities can decrypt. Symmetric encryption, known as private key cryptography, uses the same key for encryption and decryption, as illustrated in Figure 3(a). Block ciphers are the most prevalent type of symmetric encryption that divide plaintext into fixed-size blocks and encrypt them into ciphertext blocks of the same size.

Asymmetric Encryption: Asymmetric encryption, also known as *public-key cryptography*, uses two distinct keys, a *public key* and a *private key*. As shown in Figure 3(b), the public key is used to encrypt the data, while the private key is used to decrypt it. In addition, asymmetric encryption can be used to implement *digital signatures* for ensuring authenticity in communications. To this end, the sender signs data with a private key, and the receiver verifies the signature with the sender's public key. Figure 3(c) demonstrates this process.

Hash and Message Authentication Code: Hash functions maintain data integrity by converting input data of arbitrary length into unique and fixed-length hash values. As slight changes in the input result in completely different hashes, hash functions are effective for detecting any modification to the original data. **Message Authentication Codes** (**MACs**), while similar to hash functions, also incorporate a secret key. This key allows the sender to authenticate their identity as the message's origin, thereby ensuring both authenticity and integrity.

Key Derivation: A **Key Derivation Function** (**KDF**) generates a cryptographic key from a *password* or *passphrase* that fulfills standards such as minimum length, entropy, and brute-force resistance. It is commonly employed in combination with ***Password-Based Encryption (PBE)***. The process of key derivation through a KDF typically involves applying a hash function, using a random value, called *salt*, for an adequate number of *iterations* to prevent brute-force attacks.

Key Storage: Preserving the confidentiality and integrity of encrypted data in cryptography heavily relies on proper key storage practices. Key storage algorithms are designed to assist developers in securely storing sensitive credentials, such as key material. These algorithms require a strong *password* or *passphrase* as input to provide adequate security.

PseudoRandom Number Generator: Randomness plays a crucial role in all aspects of cryptography. Cryptography APIs offer **PseudoRandom Number Generator** (**PRNG**) functions to ensure the generated number holds the requisite level of randomness for cryptographic applications. PRNGs rely on a seed for generating random numbers that must also be random to prevent any potential predictability in the generated numbers.

Our survey covered 43 studies examining the usage of crypto APIs. Of these, 36 focused on the **Java Cryptography Architecture** (**JCA**), while the Python PyCrypto API and the C/C++ CommonCrypto API were each examined in 3 studies, and JavaScript and Go were each covered in 1 study.

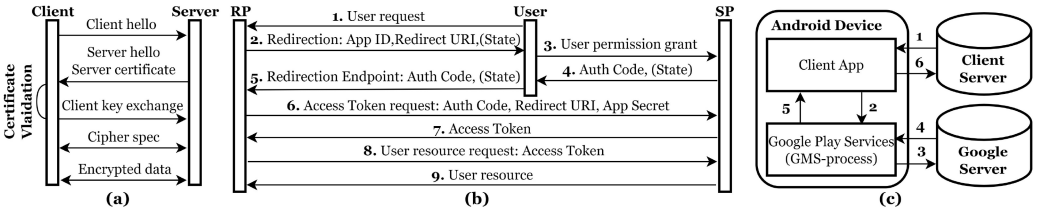


Fig. 4. (a) An overview of a simplified SSL handshake, (b) OAuth authorization code grant flow between user, RP, and SP, and (c) attestation process performed using Google SafetyNet Attestation API [S69].

4.1.2 SSL/TLS APIs. **Secure Sockets Layer** (**SSL**) and **Transport Layer Security** (**TLS**) protocols are used to establish a secure channel for communication between a client and a server, protecting them from potential attacks like a **Man-in-the-Middle** (**MitM**). These protocols rely on cryptographic primitives to ensure the authentication, confidentiality, and integrity of network messages. To establish and validate SSL connections, developers can utilize SSL/TLS APIs such as OpenSSL [29], which encapsulate the details and functionalities of these protocols. A critical aspect of SSL connection establishment is authenticating the server. During the SSL handshake, shown in Figure 4(a), the server presents its *public key certificate* to the client as a means of authentication. It is essential that the client carefully verifies the authenticity of the server's certificate to ensure the security of the SSL connection. The validation process involves ***certificate validation*** and ***hostname verification***.

In ***certificate validation***, a client must carefully verify that the certificate has been issued and signed by a trusted ***Certificate Authority (CA)*** and has not expired or been revoked. Additionally, the client must validate the certificate chain presented by the server. This involves verifying each certificate in the chain and ensuring that each one is issued by the CA immediately above it, with a trusted root CA at the top of the chain.

In ***hostname verification***, a client must verify that the *hostname* included in the certificate matches the hostname that the client is attempting to connect to. This verification process prevents MitM attacks, where an attacker intercepts the communication between a client and server, and impersonates the server by sending false information to the client.

In our survey, 26 studies focused on the usage of SSL/TLS APIs, with 22 studies for Java APIs, e.g., **Java Secure Socket Extension** (**JSSE**), and 6 studies for C/C++ APIs, e.g., OpenSSL.

4.1.3 OAuth APIs. **Open Authorization** (**OAuth**) [12] is a popular authorization protocol that allows end-users to grant third-party websites or applications access to their private resources on a remote server without sharing their credentials. This process involves three major roles: the ***user*** or ***resource owner*** who owns protected resources, the ***Service Provider (SP)*** that hosts the resources, and ***Relying Party (RP)***, known as *client application* that uses the SP to obtain access to the user's resources. ***Access tokens*** are issued by SPs to RPs with the owner's approval for accessing protected resources. OAuth APIs provided by SPs such as Google, Twitter, or Facebook are used by developers to authenticate users or obtain access to users' resources through their major accounts in SPs. Although OAuth was first introduced as an authorization framework, it has been widely adopted to implement Single-Sign-On authentication, making it difficult for developers to use it properly. To address this challenge, OpenID Connect [30] was introduced as an authentication framework based on OAuth. The OAuth specification [12] defines four different protocol flows or grant types: ***(i) authorization code***, ***(ii) implicit***, ***(iii) resource owner password credentials***, and ***(iv) client credentials***. Figure 4(b) depicts the process of authorization code grant, which is the most commonly used grant type. As shown in Figure 4(b), the process begins with a

user sending a request to an RP to access a remote resource (step 1). The RP then redirects the user to the SP with an *APP ID* and an optional *state* parameter to bind this request (step 2). Next, the user authenticates with the SP and grants the RP's requested permissions (step 3). The SP issues an *authorization code* and an optional *state* parameter to the user (step 4). The user is then redirected back to the RP's redirection endpoint, where the request is rejected if the received *state* parameter mismatches the initial one (step 5). Next, the RP sends the *authorization code* and its *secret* (established during registration with the SP) to the SP to request an access token (step 6). The SP verifies the RP app by validating the *App ID* and app *secret* and then responds with an *access token* (steps 7). With the access token, the RP requests user data from the SP, which is then shared with the RP accordingly (steps 8-9). In the implicit grant (which is simpler than the authorization code grant), in step 4, the SP directly responds with an access token instead of an authorization code, without authenticating the RP. Resource owner password credentials and client credentials grants are rarely used.

Our review included 8 studies that evaluated the usage of OAuth APIs in Android and web applications. Additionally, one study [S54] focused on both OAuth and OpenID Connect APIs for implementing authentication in Android apps.

4.1.4 Fingerprint APIs. Our review identified one study [S37] that investigated the usage of Fingerprint API in Android apps. Both Google [31] and OWASP [32] guidelines recommend using a fingerprint reader in conjunction with cryptographic operations for secure authentication. This involves using the fingerprint to unlock a cryptographic key protected by the ***Trusted Execution Environments (TEE)***, rather than just recognizing the user. TEE, an integral part of modern smartphones, can securely generate and store cryptographic keys. Combining TEE with fingerprint readers for Two-factor authentication provides strong security comparable to external hardware devices such as YubiKeys [33]. To interact with the fingerprint sensor and verify whether a legitimate user has touched it, four essential steps are required to follow [S37]. That begins with ***generating a cryptographic key*** where developers specify key properties via parameters such as setting the *user authentication required* parameter to *True*, ensuring key usability only after a legitimate user has touched the fingerprint reader. Next, ***the key is unlocked through user authentication***. If a legitimate user touches the sensor, the cryptographic key is unlocked, triggering a series of callback functions. Developers can ***override the fingerprint callbacks*** to handle different scenarios based on user legitimacy. Once authenticated, ***the unlocked key can be used*** by an app to encrypt, decrypt, or sign data. Google recommends using a previously generated private key to sign a server-provided authentication token to authenticate, and then to send this token to the app's remote backend [31].

4.1.5 Spring Security APIs. Spring Security [34] is a powerful and highly customizable framework for securing Java-based applications. It provides a wide range of security services, such as authentication, authorization, and access control. While Spring Security provides its own set of authentication features, it also supports integration with various authentication mechanisms such as ***Lightweight Directory Access Protocol (LDAP)***, *OAuth*, and ***Java Database Connectivity (JDBC)***. The framework enables developers to implement role-based and permission-based authorization, allowing for granular access control policies for different parts of their applications. It allows the implementation of an access-control specification model by typically defining various filters for finding access to a given resource. In addition, it offers features for securing communication between different application components using HTTPS, SSL/TLS, and other encryption mechanisms. In our review, one study [S38] examined Spring Security's use for implementing an access-control specification model.

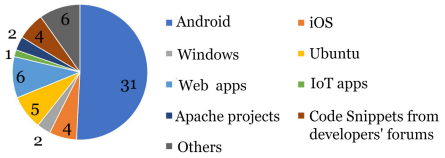


Fig. 5. Distribution of types of software artifacts analyzed by reviewed studies.

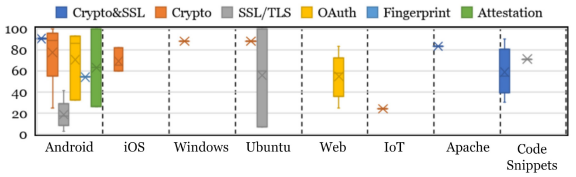


Fig. 6. Boxplots with mean markers illustrating the percentage of software artifacts with at least one misuse (misuse rate).

4.1.6 SafetyNet Attestation APIs. Attackers can alter an application's behavior either by directly modifying the app or by obtaining root access to the host system and injecting malicious code. Hence, developers need to ensure their app's code integrity and the client device's status. Google offers the SafetyNet Attestation API [35] to check the integrity of a device or application and detect compromised devices and tampered applications. Figure 4(c) illustrates the attestation process using this API. The *attest* function, requiring a *nonce* and an *API Key*, triggers the attestation API. The nonce is generated by the application's backend server and sent to the device upon attestation request (step 1). The API Key is created using Google's *API Console*, a platform for developers to manage their Google APIs. When attestation is requested (step 2), ***Google Mobile Services (GMS)*** conducts several checks on the device, forwarding the results to Google's server (step 3). Google's server responds with signed attestation data (step 4), which GMS delivers to the client (step 5). The client app extracts a ***JSON Web Signature (JWS)*** from the data, sending it to the backend server for validation (step 6), followed by the client server verifying the JWS (step 7). In our review, one study [S69] analyzed the usage of the attestation API in Android applications.

4.2 Misuse Trends

A total of 50 primary studies have investigated real-world software to identify security API misuse. This section provides insights into their findings (Section 4.2.1) and analyzes the variations observed (Section 4.2.2).

4.2.1 Security API Misuse. Figure 5 demonstrates diverse types of software analyzed and their distribution over reviewed studies. Given the widespread use of mobile devices for storing confidential data and Android being the most widely used OS, Android apps have received the most significant attention from researchers studying security API misuse. Many studies used the Google Play Store as the primary source for obtaining Android apps. Several studies also used AndroZoo [36], which offers the most extensive publicly available dataset of Android apps collected from the Google Play Store. Researchers also analyzed iOS apps from the official Apple App Store. Additionally, two studies leveraged Maven Central and GitHub to analyze Apache projects, and one study [S22] assessed 521 IoT firmware obtained from various IoT vendors' websites. There were also studies dedicated to assessing Windows and Ubuntu applications, and studies that investigated OAuth API usage in web applications. In addition, four studies examined the state of security API usage in code snippets found in developers' forum posts. This holds significance as prior research revealed that forums like **StackOverflow** (**SO**) are crucial sources of information for developers, who may use code snippets from such sources in their software projects [37]. Three studies assessed 187 [S33], 3,834 [S45], and 25,855 [S66] code snippets from SO, while another study [S64] investigated 140, 71, and 48 posts from **Oracle Java Cryptography** (**OJC**), **Google Android Developers** (**GAD**), and **Google Android Security Discussions** (**GASDs**), respectively.

The findings from these studies reveal a concerning trend of security API misuse among developers, with some studies reporting misuses in nearly 100% of the applications examined. For example, one study [S11] analyzed ten thousand Android apps and found that 95% of them contained at least one crypto misuse. Similarly, another study [S45] exploring a dataset of Android applications discovered that 15% of these apps incorporated security-related code snippets from Stack Overflow, of which 98% were found to contain misuses. Figure 6 summarizes the findings, illustrating misuse rates for each API across various software types. Misuse rate, as defined here, represents the percentage of software artifacts identified with at least one security API misuse.

4.2.2 Variations in Findings. As shown in Figure 6, there are wide ranges of misuse rates for some APIs. For instance, crypto misuse rates in Android applications range dramatically from 25% to 100%. Even code snippets from developer forums exhibit a wide range of misuse rates (48%–90%). Similar variations are observed for other APIs and platforms, such as SSL misuse rates in Ubuntu, which vary significantly from 7% to 100%. There are various reasons for these substantial differences. We identified five key factors contributing to these variations, including ***dataset size, data source, temporal consideration, misuse scope***, and ***detection techniques***.

First, the size of the datasets analyzed in each study can vary considerably. Android studies, for example, encompass investigations targeting a mere 45 applications [S15] to massive analyses exceeding 500,000 apps [S36]. Second, the data for analysis has been collected from various sources. For instance, the study exploring developer forums, [S64], reported differing misuse rates–90% in OJC, 71% in GAD, and 48% in GASD, reflecting the diverse nature of these platforms. It is important to consider that GASD is not exclusively focused on programming, potentially contributing to the lower misuse rate observed. Third, studies were conducted across different years and utilized datasets collected at various time intervals. Security practices evolve over time, potentially affecting misuse prevalence. For example, a study [S12] compared datasets from 2012 and 2016 to understand changes in misuse prevalence. Their analysis revealed a decrease in specific misuse types, likely due to increased developer awareness of insecure practices over time. Conversely, new misuses might be discovered over time, leading to apparent increases in misuse rates. Fourth, studies also differ in their focus and scope of misuse. Some delve into a single, specific misuse, while others aim for a broader analysis. Additionally, the prevalence of different misuse types naturally varies. The specific types and prevalence of misuses identified will be discussed in detail in Section 5. Last but not least, the detection techniques employed in these studies can introduce variations in the reported misuse rates. Some techniques may have limitations, such as reporting false positives or non-exploitable misuses. For example, a study [S69] initially reported a 100% misuse rate of SafetyNet Attestation in Android apps integrating this API, but after conducting exploit attempts, it found that only a small set of apps had exploitable misuses. Section 6 will dive deeper into the details of detection techniques.

Despite the varied findings, it is evident that security API misuse remains a significant challenge for developers, and our study represents a timely effort to raise awareness about this critical aspect of secure software development and to provide fundamental knowledge for the development of effective detection approaches.

5 RQ2: Misuses of Security APIs

Through thorough analysis and semantic categorization, we identified 30 distinct misuse types, each assigned a unique identifier (*M#*). Table 4 lists these misuses and their mappings to relevant security APIs, while Figure 7 illustrates the number of research efforts dedicated to each misuse type. As shown, *insecure cryptographic key management* (*M1*), *insecure cryptography algorithms* (*M4*), and *improper SSL certificate validation* (*M8*) are the most extensively explored misuses in

Table 4. Taxonomy of Security API Misuses and Mappings with Security APIs (C for Cryptography, T for SSL/TLS, O for OAuth, F for Fingerprint, S for Spring Security, and A for SafetyNet Attestation)

Misuse	C	T	O	F	S	A	Misuse	C	T	O	F	S	A
M1) Insecure cryptographic key management	✓	✓	-	✓	-	-	**M16)** Lack or misuse of authentication	-	-	✓	✓	✓	-
M2) Insecure storage of credentials	✓	✓	✓	-	-	-	**M17)** Insecure OAuth grant types	-	-	✓	-	-	-
M3) Insecure cryptographic PRNGs[1]	✓	✓	-	-	-	-	**M18)** Lack of PKCE parameters in OAuth	-	-	✓	-	-	-
M4) Insecure configurations for encryption	✓	✓	-	-	-	-	**M19)** Lack or misuse of cryptography	-	-	-	✓	-	-
M5) Insecure configurations for PBE[2]	✓	✓	-	-	-	-	**M20)** Lack of authorization	-	-	-	-	✓	-
M6) Insecure cryptography algorithms	✓	✓	-	-	-	-	**M21)** Incorrect authorization	-	-	-	-	✓	-
M7) Improper SSL hostname verification	✓	✓	-	-	-	-	**M22)** Spring method call with higher access rights	-	-	-	-	✓	-
M8) Improper SSL certificate validation	✓	✓	-	-	-	-	**M23)** Local attestation checks	-	-	-	-	-	✓
M9) Improper SSL socket	✓	✓	-	-	-	-	**M24)** Sending incomplete data to attestation server	-	-	-	-	-	✓
M10) Insecure SSL/TLS standard	✓	✓	-	-	-	-	**M25)** Local nonce generation for attestation	-	-	-	-	-	✓
M11) Improper error handling	✓	✓	-	-	-	-	**M26)** Verification flaws in attestation	-	-	-	-	-	✓
M12) Lack of SSL protection	✓	✓	✓	-	-	-	**M27)** Using test server for attestation	-	-	-	-	-	✓
M13) Insecure *state* management in OAuth	-	-	✓	-	-	-	**M28)** Null or wrong API key	-	-	-	-	-	✓
M14) OAuth client-side API call	-	-	✓	-	-	-	**M29)** Using deprecated API	-	-	-	-	-	✓
M15) Insecure OAuth redirection options	-	-	✓	-	-	-	**M30)** Performing attestation only at first launch	-	-	-	-	-	✓

(1). Pseudorandom Number Generators, (2). Password-Based Encryption

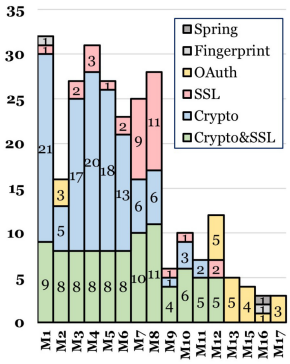


Fig. 7. Number of studies for misuses grouped by APIs (including only misuses with multiple studies).

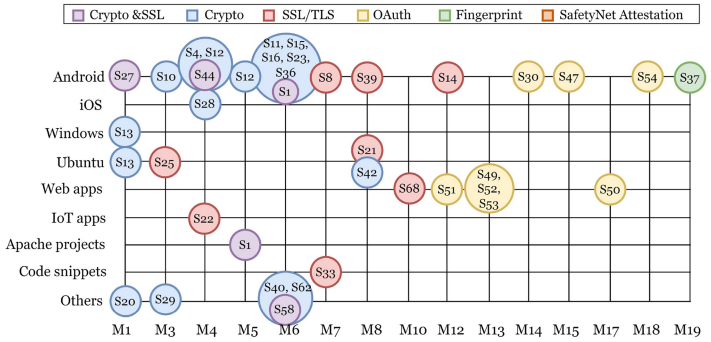


Fig. 8. Most common security API misuses in real-world software identified by primary studies.

research. Notably, they are also among the most common misuses reported for crypto and SSL APIs. Figure 8 illustrates different findings regarding the most common misuses of security APIs identified in real-world software by reviewed studies. *Insecure cryptography algorithms (broken hash)* for crypto API, *improper certificate validation* for SSL/TLS API, *lack or misuse of the State parameter (M13)* for OAuth API, and *lack or misuse of cryptography (M19)* for Fingerprint API are the misuses seen with the highest frequency in this figure. In the following, we elaborate on these misuse types, discussing their consequences and prevalence in software applications.

M1–Insecure cryptographic key management: M1 is the most extensively studied misuse in the literature, specifically on ***crypto***, ***SSL/TLS***, and ***Fingerprint*** APIs (Figure 7). It also ranks among the most common misuses found in various software applications across Android, Windows, and Ubuntu (Figure 8). In the context of crypto and SSL/TLS, M1 often arises from practices like *deterministic key generation* and *poor key derivation*, leading to predictable and easily compromised keys. The use of *hard-coded, static (constant), expired, previously-used* or any kind of *predictable* keys is among the misuses identified by prior research. Listing 1 illustrates an example of hard-coded keys identified within an Android app. Keys of *inadequate length* also increase the possibility of a brute-force attack; for example, RSA keys and ECC keys should be at least 2048 and 224 bits long, respectively [38]. Additionally, the practice of *retaining keys in memory* after their intended

use creates opportunities for code injection or side-channel attacks, enabling potential key recovery [S13]. Another significant concern arises from *insecure key distribution*. Using a key agreement protocol that allows a single peer to generate the shared secret without involving the other peers compromises the security of the negotiated key [S13]. Likewise, *key exchange without authentication* from a trusted entity leads to vulnerabilities to MitM attacks, where malicious servers can impersonate trusted servers and gain access to sensitive information [S15]. Furthermore, *ignoring integrity checks* is a notable concern. Developers should conduct integrity checks after symmetric key exchange to ensure the integrity of the exchanged keys [S29]. In the context of ***Fingerprint***, M1 often arises from a *lack of cryptographic key generation* when developers neglect using methods for generating keys to be used for securing fingerprint-based authentication [S38]. Another concern is the *use of unlocked keys*, where the key remains unlocked, allowing attackers with root access to use it without requiring the user to touch the fingerprint sensor [S38].

```
1  public class AESUtil {
2    String iv = "0102030405060708";
3    String key = "czabcd1234czabcd";
4
5    String Encrypt(String sSrc) throws
     Exception {
6      if (key == null || key.length() != 16) {
7        return null;}
8
9      byte[] raw = key.getBytes("UTF-8");
10     SecretKeySpec s = new SecretKeySpec(raw, "AES");
11     Cipher ci = Cipher.getInstance("AES/CBC/
     Iso10126Padding");
12     IvParameterSpec ps;
13     ps = new IvParameterSpec(iv.getBytes("UTF-8"));
14     ci.init(Cipher.ENCRYPT_MODE, s, ps);
15     byte[] encrypted = ci.doFinal(sSrc.
     getBytes("UTF-8"));
16     return new String(Base64.encodeBase64(
     encrypted),"UTF-8");}}
```

Listing 1. M1 and M4 identified in an Android app [S19].

```
1  protected String getKeystorePassword() {
2      String keyPass = (String)attributes.get("keypass");
3      if (keyPass == null) { keyPass = "changeit";}
4      String keystorePass = (String)attributes.get("keystorePass")
       ;
5      if (keystorePass == null) { keystorePass = keyPass;}
6      return keystorePass;}
```

Listing 2. M2 within a popular application-server artefact [S11].

```
1  void onCreate(Bundle savedInstanceState) {
2      super.onCreate(savedInstanceState);
3      setContentView(R.layout.activity_main);
4      SecretKeySpec secretKeySpec = null;
5      SecureRandom sr = SecureRandom.getInstance("SHA1PRNG");
6      sr.setSeed("any data used as random seed".getBytes()); ...}
```

Listing 3. M3 identified in an Android app [S19].

M2–Insecure storage of credentials: Several studies examining ***crypto***, ***SSL/TLS***, and ***OAuth*** APIs highlight a concerning trend: developers often compromise security by using insecure storage practices for credentials (passwords, secret keys, etc.). Storing credentials in plain text *strings* leaves them vulnerable to credential dumping as programming languages like Java do not clear string values from memory unless the garbage collector runs [39]. Similarly, storing sensitive data in *files* or *shared storage* poses a significant risk of data exposure. It is recommended to use the secure `key storage` method offered by crypto APIs, which requires a password for access [40]. However, developers often undermine its security by using *hard-coded, static (constant), or predictable passwords* (Listing 2). In the context of OAuth, M2 arises from the *local storage of RP secrets and access tokens* [S30, S48]. Attackers can reverse engineer client-side applications to steal locally stored RP secrets and impersonate legitimate applications to request access tokens from SPs. Stolen access tokens can also grant unauthorized access to user accounts and sensitive information stored on SPs.

M3–Insecure PseudoRandom Number Generators (PRNGs): Insecure `PRNGs` are a major source of ***crypto*** and ***SSL/TLS*** vulnerabilities [41]. It is essential to exclusively use the secure `PRNG` offered by crypto APIs and use a truly randomly generated seed for initialization. However, developers often make two common mistakes: (1) using *simple* `PRNGs` that are known to be insecure as they can generate predictable random numbers [42], and (2) using *static (constant), low-entropy, predictable or previously-used* seeds, as depicted in Listings 3 and 5. These practices severely undermine the security of cryptographic materials, like keys, that rely on randomness for their strength.

M4–Insecure configurations for encryption: One common misuse of ***crypto*** and ***SSL/TLS*** APIs is using *unsafe modes of operation* for encryption, such as **`Electronic Codebook (ECB)`**. `ECB` mode

encrypts data blocks independently, transforming identical message blocks into identical ciphertext blocks, thus revealing data patterns and compromising confidentiality. To ensure security, it is recommended to use more secure modes, such as `Cipher Block Chaining (CBC)` or **`Galois/Counter Mode (GCM)`**. Other instances of unsafe encryption modes include `DESede` with `ECB`, `DES` with `CBC3 SHA`, `AES` with `CBC` and `PKCS5Padding`, `CBC` without `HMAC`, and `3DES` with `EDE CBC SHA`. Additionally, ***Initialization Vectors (IVs)*** are crucial in several encryption modes to add entropy to ciphertexts. To ensure the security of cryptographic schemes, IVs must be randomly and properly generated. However, some developers introduce vulnerabilities by using *empty, zeroed, hard-coded, static, badly-derived (e.g., deriving from keys or messages), short-length, previously-used, or any kind of predictable IVs*. Listing 1 illustrates an example of hard-coded IVs identified within an Android app. Another parameter that requires secure configuration is the *padding* scheme, which specifies how to fill the last block of data in encryption. Missing padding or using insecure padding (e.g., `PKCS 1-v1.5` for `RSA`) makes it easier for an attacker to launch a padding oracle attack and recover the plaintext.

M5–Insecure configurations for Password-Based Encryption (PBE): Within ***crypto*** and ***SSL/TLS*** APIs, `PBE` relies on carefully chosen parameters including *salt*, *password*, and *iteration count* to derive strong encryption keys. Improperly setting these parameters can significantly compromise the security of the derived key. Common misconfigurations of `PBE` include using an *empty, static (constant), short-length (size<64 bits [43]), or predictable salt* or using a *hard-coded, static, weak, NIST-blacklisted, expired, previously-used, or predictable password*, which introduces vulnerabilities to brute-force and dictionary attacks. Moreover, developers may prefer to choose *small iteration counts* (less than 1,000 [43]) to achieve better performance, making it easier for attackers to perform brute-force attacks. Listing 4 showcases M5 present in an Android application. The code uses an iteration count based on the password length, falling far below the recommended minimum. Additionally, the salt is generated by hashing the password itself, which introduces another security risk. An attacker gaining access to the salt could potentially recover the password [S1].

M6–Insecure cryptography algorithms: Algorithms once considered secure can become vulnerable due to newly discovered weaknesses and attacks. This makes it challenging for developers to keep up with the latest updates. Thus, one of the most common misuses of ***crypto*** and ***SSL/TLS*** APIs involves using outdated algorithms including: *unsafe symmetric encryption algorithms such as 64-bit block ciphers (*e.g., `DES`, `IDEA`, `Blowfish`, `RC4`, `RC2`*), weak PBE algorithms (*e.g., `PBKDF1`*), insecure asymmetric ciphers (*e.g., `RSA`, `ECC`*), insecure cryptographic MACs and broken hash functions (*e.g., `SHA1`, `MD5`, `MD4`, `MD2`*) as well as insecure combinations of encryption and hashes or MACs (*e.g., `PBKDF` *with* `SHA224`*)*. Listing 5 shows the use of insecure M5 for hashing found in a code snippet from developers' forums.

```
1 PBEKeySpec getPBEParameterSpec(String password) throws Throwable {
2     MessageDigest md = MessageDigest.getInstance(MD_ALGO); // MD5
3     byte[] saltGen = md.digest(password.getBytes());
4     byte[] salt = new byte[SALT_SIZE];
5     System.arraycopy(saltGen, 0, salt, 0, SALT_SIZE);
6     int iteration = password.toCharArray().length + 1;
7     return new PBEKeySpec(password.toCharArray(), salt, iteration);}
```

Listing 4. M5 identified in an Apache project [S1].

```
1     // weak hash of user's password
2     md = MessageDigest.getInstance("MD5");
3     byte[] hash = md.digest(password.getBytes());
4     // weak PRNG with fixed seed
5     sr = SecureRandom.getInstance("SHA1PRNG");
6     sr.setSeed(hash.getBytes());
7     byte[] keyBytes = new byte[]
```

Listing 5. M3 and M6 in a code snippet from developers' forums [S64].

M7–Improper SSL hostname verification: Hostname verification is a crucial security measure that ensures the hostname in the SSL certificate matches the server hostname to which the client is trying to connect. However, various studies in the context of ***crypto*** and ***SSL/TLS*** APIs [e.g., S1, S2, S8, S9] have revealed that some developers *trust all hostnames* or do not verify the hostname correctly. For instance, the code snippet in Listing 6, sourced from SO, always returns true, thereby

accepting all hostnames. Improper hostname verification enables an attacker to intercept the communication between the client and the server by presenting a valid SSL certificate for a poisoned hostname.

M8– Improper SSL certificate validation: Many developers make mistakes in implementing proper certificate validation as identified by several studies in the context of ***crypto*** and ***SSL/TLS*** APIs [e.g., S14, S15, S19]. One of the most common mistakes is blindly *trusting all certificates*, allowing attackers to present fake certificates and gain unauthorized access to sensitive information. An example is shown in Listing 7, where validation is implemented through empty methods. Additionally, some developers only check that each certificate in the chain has not expired without performing any other validation. Other ways of compromising certificate validation include *incomplete validation, neglecting to check for expiration or revocation, trusting self-signed certificates, trusting too many CAs, trusting certificates with unclear names, inadequate CA verification, or insecure certificate pinning*.

```
1  //Empty HostnameVerifier - Accepts all hostnames
2  public boolean verify(String hostname, SSLSession session) {
3      return true;
4  }
```

Listing 6. M7 in a code snippet from StackOverflow [S45].

```
1  class r\$b implements X509TrustManager {
2      public void checkClientTrusted(X509Certificate[], String) {}
3      public void checkServerTrusted(X509Certificate[], String) {}
4      public X509Certificate[] getAcceptedIssuers() { return null;}}
```

Listing 7. M8 identified in an Android app [S43].

M9–Improper SSL socket: In the context of ***SSL/TLS*** API, the SSL socket is aimed at establishing a connection between a specific host and a specific port. Nonetheless, verifying and authenticating the server's hostname is essential before establishing the connection. A flawed implementation of the SSL socket may ignore hostname verification when creating the socket, as depicted in Listing 8 [S1].

M10–Insecure SSL/TLS standard: TLS, the successor of SSL, is generally considered to be more secure. However, older versions of TLS, including TLS 1.0 and TLS 1.1, have been found to be susceptible to various types of attacks, such as POODLE, BEAST, and CRIME, and therefore are no longer deemed secure. These outdated versions have been deprecated [44–46], and TLS 1.2 is being recommended as the minimum protocol version for secure communication. Nevertheless, several studies on ***crypto*** and ***SSL/TLS*** APIs have revealed that some developers still use outdated versions of TLS and compromise the security of transmitted data [S2, S66, S68, etc].

M11–Improper error handling: Some developers prioritize functionality over security, leading them to disregard errors, as identified in use cases of the ***crypto***, ***SSL/TLS***, and ***SafetyNet Attestation*** APIs. For crypto and SSL/TLS, developers may ignore errors occurring during certificate validation and simply proceed with normal operations [S24, S34, S43, etc.]. Listing 9 exemplifies this by using an empty method for exception handling in an Android app. Likewise, errors may occur during the integrity checks performed by the SafetyNet Attestation API, and disregarding them can result in the failure of the attestation process [S69].

```
1  try {   SSLContext instance = SSLContext.getInstance("TLS");
2          // ...
3          this.webSocketClient.setSocket(instance.getSocketFactory
       ().createSocket());
4  } catch (Throwable e) { ... }
5  this.webSocketClient.connect();
```

Listing 8. M9 identified in an Android app [S1].

```
1  void checkServerTrusted(X509Certificate[] chain, String str){
2      try {
3          this.f7427a.checkServerTrusted(chain, str);
4      }
5      catch (CertificateException e) {} //Ignores exception
6  }
```

Listing 9. M11 identified in an Android app [S1].

M12–Lack of SSL protection: M12 represents a prevalent misuse identified for ***crypto***, ***SSL/TLS***, and ***OAuth*** APIs. While using crypto and SSL/TLS, *occasional use of HTTP* exposes the

application to potential attacks like SSL stripping [47, 48], wherein a malicious actor can launch a MitM attack on an SSL connection [e.g., S1, S5, S9, S10]. For OAuth, security heavily relies on the secure transmission of messages throughout the process. *Transmitting messages in plaintext without SSL/TLS encryption* enables attackers to eavesdrop and pilfer access tokens or other OAuth credentials [S30, S47, S50, S51]. Listing 10 shows the use of a raw access token for OAuth transmissions in an Android app.

```
String aToken = getAccessToken();
HttpClient httpClient = new DefaultHttpClient();
HttpPost httpPost = new HttpPost("/backend.com/tokensignin");
List<NameValuePair> params = new ArrayList<>(1);
params.add(new BasicNameValuePair("access_token", aToken));
httpPost.setEntity(new UrlEncodedFormEntity(params));
httpClient.execute(httpPost);
```

Listing 10. M12 (raw OAuth access token) in an Android app [S30].

```
this.getAuthUrl = function () {
    var scopes = ['wl.signin', 'wl.basic', 'wl.offline_access'
    , 'office.onenote_create'];
    var query = toQueryString({
        'client_id': clientId,
        'scope': scopes.join(' '),
        'redirect_uri': redirectUrl,
        'display': 'page',
        'locale': 'en',
        'response_type': 'code' });
    return oauthAuthorizeUrl + "?" + query;};
```

Listing 11. M13 (no state parameter) in example code for connecting to Microsoft's Live Connect Services [S52].

```
String url = "api.provider.com/..";
HttpURLConnection c=(HttpURLConnection) new URL(url).openConnection();
c.setRequestMethod("POST");
// ...
List<String> params = new ArrayList<>();
params.add("oauth_consumer_key=" + client_id);
params.add("oauth_token=" + access_token);
params.add("oauth_signature=" + getSignature(client_secret));
// ...
c.setRequestProperty("Authorization", createHeaders(params));
String user_id = parseJSON(c.getInputStream(), "id");
newUserLogin(user_id);
```

Listing 12. M14 identified in an Android app [S30].

```
WebView webview;
String url = "provider.com/..?client_id=".."&redirect_url=".."
    &response=code";
...
public void onCreate() {
    webview.loadUrl(url);
}
```

Listing 13. M15 identified in an Android app [S30].

M13–Insecure state management in OAuth: The `state` parameter protects user sessions in ***OAuth*** transactions against **Cross-Site Request Forgery** (**CSRF**) attacks by verifying request authenticity. In CSRF, an attacker uses a user's previous session data to make a malicious request on their behalf [S52]. OAuth guidelines recommend generating and validating a randomized `state` parameter, bound to the user's session to prevent such attacks [12]. However, developers may misunderstand its purpose leading to mistakes such as *using a constant or predictable value, enabling multiple replays, neglecting* `state` *parameter verification, accepting requests without a* `state` *parameter, or assuming that all* `state` *parameters generated by their app are valid without proper session binding checking* [S51]. Listing 11 illustrates an example where OAuth is implemented without using a `state` parameter. This example is provided by Microsoft to help developers use their API. Nonetheless, it can mislead developers who copy the example without understanding the security implications.

M14–OAuth client-side API call: A significant security concern in ***OAuth*** authentication flows arises from the reliance on client-side API calls, which attackers could easily manipulate [S30]. Listing 12 presents a code snippet from an Android app wherein an access token is exchanged for a user ID via an API call from the app to authenticate users.

M15–Insecure OAuth redirection options: To ensure the security of ***OAuth*** transactions, it is crucial to use secure methodologies for handling redirection [12]. Insecure redirection methods can allow attackers to redirect users to arbitrary domains or URLs, potentially leading to further attacks or data theft. For instance, in a mobile context, using WebView is considered insecure as it undermines the isolation between an SP and an RP [S30]. A malicious RP can use the WebView of their mobile applications to host an SP, allowing them to access the user's cookies and log in on the user's behalf. Listing 13 exemplifies M15 with the use of WebView for redirection in an Android application.

M16–Lack or misuse of authentication: This misuse has been identified in use cases of ***OAuth***, ***Fingerprint***, and ***Spring Security*** APIs. In OAuth transactions, an SP is responsible for authenticating an RP, and vice versa. However, a study [S47] on a collection of Android apps revealed that none of the RP apps in their investigation verified the SP's identity. Furthermore, M16 was

observed in the context of the Fingerprint API, where developers designate authentication methods as null [S37]. Additionally, research by [S38] found M16 in Spring Security usage, where developers neglect to implement authentication for accessing a resource. This can occur if a developer forgets to include an authentication filter or improperly configures a filter, allowing unrestricted access to the resource (CWE-306) [49].

M17–Insecure OAuth grant types: The security of ***OAuth*** transactions highly depends on the choice of grant type. It is essential to avoid using insecure grant types, such as implicit for authentication. Implicit grants raise a major security concern because the access token is not bound to the intended RP, which enables an attacker to use a user's access token, issued to the malicious application, to log in as the user on a benign application [S48]. Best current practices recommend using the authorization code flow, which can be protected by PKCE [S54].

M18–Lack of PKCE parameters for OAuth authorization code grant: The authorization code grant is generally considered to be the most secure grant type for ***OAuth***. However, it remains susceptible to code interception attacks, where an attacker intercepts the authorization code sent by the SP and uses it to obtain an access token [50]. To mitigate this vulnerability, the **Proof Key for Code Exchange** (**PKCE**) protocol was introduced in the OAuth 2.0 specification [51]. PKCE verifies that the requesting application is the same one that originally requested the authorization code by using a cryptographically linked code verifier and code challenge exchanged between the application and the SP. PKCE is recommended as a mandatory security measure for public clients to secure the authorization code grant.

M19–Lack or misuse of cryptography in fingerprint authentication: This misuse occurs while using ***Fingerprint*** API when developers do not utilize any cryptography operation after the user touches the sensor or perform an insecure cryptography operation using constant encryption keys [S37].

M20–Lack of authorization: While using ***Spring Security***, a developer may fail to include appropriate authorization filters for a particular resource, which needs valid authorization according to the access-control specification model. Using authentication as the authorization filter also results in the same misuse since it only verifies the user's identity. Both scenarios lead to the vulnerability known as missing authorization (CWE-862) [52], allowing unrestricted access to resources, either for all users or just authenticated users, based on the type of applied filter [S38].

M21–Incorrect authorization: This misuse arises from an incorrect authorization formula while using an authorization filter in ***Spring Security*** [S38]. The misuse, known as the vulnerability of incorrect authorization (CWE-863) [53], results in unauthorized users gaining access to resources.

M22–Spring method call with higher access rights: M22 is another instance of the incorrect authorization vulnerability (CWE-863). It arises when a developer properly configures a resource while using ***Spring Security***, yet calls a method requiring higher access rights in a deeper application layer, which should not be accessible to the user [S38].

M23–Local attestation checks: The ***SafetyNet Attestation*** API returns a JWS object representing the device and application state. It is crucial to send the JWS object to the backend server for verification. Performing local checks enables an attacker to bypass the verification by modifying the application [S69].

M24–Sending incomplete data to attestation server: The ***SafetyNet Attestation*** JWS should be sent to the server for verification. However, a developer may choose to send only certain values extracted from the JWS object. This enables attackers to replace the missing values on a compromised device or application without any means for servers to detect tampering [S69].

M25–Local nonce generation for attestation: SafetyNet Attestation relies on nonces used in the `attest` function to prevent replay attacks. Nonces are included in the JWS output and checked against the value passed to the function to confirm that the correct JWS result is being attested.

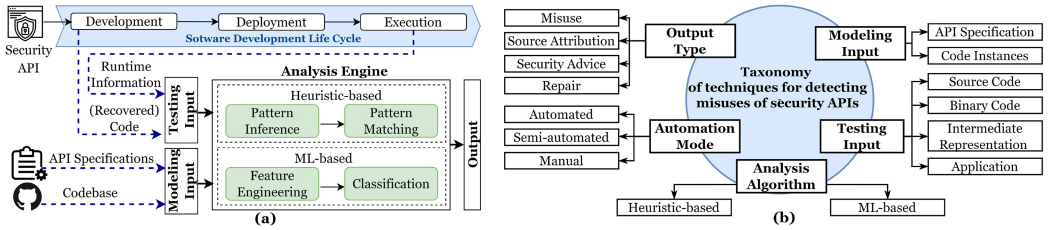


Fig. 9. (a) Overview and (b) taxonomy of techniques used by primary studies for detecting misuses of security APIs.

However, if the nonce value is generated locally on a compromised device or application, an attacker can exploit a previously generated nonce value to conduct a replay attack [S69].

M26–Verification flaws in attestation: The verification process of ***SafetyNet Attestation*** JWS involves several checks by the server, such as validating the nonce, APK package name, and the hash of the application's signing certificates present in the JWS payload. Inaccurate or incomplete execution of these validations may enable an attacker to send a tampered SafetyNet JWS to the server and bypass the verification [S69].

M27–Using test server for attestation: Google provides a verification service for ***SafetyNet Attestation***, which is essentially a test server that allows a client application to submit a JWS for verification. It is important to note that this service is exclusively designed for testing purposes, and using it in a production environment may compromise the security of SafetyNet Attestation [S69].

M28–Null or wrong API key: Developers must provide the ***SafetyNet Attestation*** API with a valid key obtained from the Google APIs Console. However, it is not uncommon for developers to mistakenly use an incorrect or null API key, leading to an error in the attestation process [S69]. If this error is not handled properly, the attestation process fails, leaving any tampering undetected.

M29–Using deprecated API: The attestation process cannot be accomplished if developers use the deprecated ***SafetyNet Attestation*** API, which always returns an error and can not generate a valid SafetyNet JWS [S69].

M30–Performing attestation only at first launch: SafetyNet Attestation should be consistently performed during an application life cycle, specifically when launching or handling sensitive information. However, some developers only perform SafetyNet Attestation during the first launch, leaving the application vulnerable to tampering [S69]. An attacker could exploit this by initially launching the application in a non-tampered state, then subsequently tampering with the device or application without detection, as SafetyNet Attestation would no longer be performed.

6 RQ3: Misuse Detection Techniques

Figure 9(a) introduces a high-level taxonomy for misuse detection techniques employed by reviewed studies. It identifies four key components: ***Modeling Input***, the data used to build the detection model (including API specifications or code examples), ***Testing Input***, the data used to test the software for identifying misuses (including code or runtime information), ***Analysis Engine***, the core for identifying misuses that can be implemented manually, semi-automatically, or fully automated using heuristics- or **Machine Learning** (**ML**)-based algorithms, and ***Output***. Building upon this, Figure 9(b) categorizes existing literature based on these factors: Modeling Input, Testing Input, Output Type, Automation Mode, and Analysis Algorithm. The following subsections will explore each factor in detail.



Fig. 10. Distribution of misuse detection techniques adopted by primary studies over (a) modeling input types, (b) testing input types, (c) output types, and (d) automation mode.

6.1 Types of Modeling Input

Misuse detection models are built using either ***API specifications*** (63 studies) or ***code examples*** (3 studies) (Figure 10(a)). The most commonly used source is API specifications, which can be found in API documentation, relevant literature on security API misuse, or established standards from organizations like NIST and IETF [S10]. API specification-based detection involves manually defining misuse or normal patterns for security APIs and subsequently applying a pattern-matching technique to identify misuses. However, a key challenge lies in the evolving nature of API specifications. For instance, the SHA-1 hash function, once considered secure, is no longer recommended due to discovered vulnerabilities [54]. Manual patterns defined using outdated specifications might miss new or evolving threats.

To address this challenge, recent research has explored code examples as an alternative source for inferring usage patterns. Given that code repositories are regularly updated to integrate security fixes, CryptoChecker introduced DiffCode to infer crypto misuse patterns from code changes [S29]. This involves three steps: (i) collecting code changes from GitHub repositories, focusing on patches for classes that use the target API; (ii) creating **Directed Acyclic Graphs** (**DAGs**) to represent the invoked APIs and related parameter values, and comparing them to extract relevant changes; and (iii) clustering similar changes to identify API misuse patterns. On the other hand, Seader [S2] and VuRLE [S46] leveraged **Abstract Syntax Trees** (**ASTs**) of a given vulnerable code and its corresponding repaired code to infer misuse patterns from code examples. Additionally, source code from repositories was used to train ML-based detection models to classify API use within an application as either normal or misuse [S18, S45, S65].

6.2 Types of Testing Input

Various input types are utilized to analyze an application for the presence of misuse, including ***source code, binary code, intermediate representation,*** or ***applications***, depending on the analysis algorithm employed (Section 6.5 details analysis algorithms). The most common approach is analyzing the source code, which was employed in 33 studies. Some misuse detection tools are developed as IDE plugins to provide real-time feedback as developers write code. For example, FixDroid [S5] was introduced as a plugin for Android Studio, or CogniCrypt [S3] as an Eclipse plugin. However, source code analysis has limitations. Existing approaches are often limited to specific programming languages. A study found developers frequently use C for cryptography in MicroPython projects, highlighting the need for multi-language tools [S20]. Additionally, source code analysis is typically restricted to open-source applications. Some studies [S14, S23, S26] decompiled binary code to gain access to source code when the source code is unavailable. However, obfuscation techniques that modify class files to protect the source code present challenges to successful decompilation.

In the absence of source code, 25 studies employed binary code analysis. Binary files are typically packaged with other resources, such as images and configuration files, into application files. Thus, misuse detection techniques typically require *reverse engineering* to decode an APK file and disassemble it into binary files, ready for analysis. Disassembling is typically faster than

decompiling and is not affected by obfuscation. Common program analysis frameworks used by primary studies to extract binary files from application files include SOOT [55], APKTool [56], and Androguard [57].

However, analyzing low-level representations of applications (i.e., binary files) can be laborious and error-prone. Several studies [S4, S11, S12, S28, S37, S44] addressed this by converting binary files into a higher-level **Intermediate Representation** (**IR**) that accurately captures the program behavior. Using IR also helps address the inconsistency of different input formats. For instance, CryptoREX [S22] converts diverse binary files into a unified IR to carry out large-scale crypto misuse detection for IoT firmware with diverse underlying architectures. Another study [S45] detected misuse by identifying instances of code snippets with security API misuse in Android apps. To achieve this, both known vulnerable code and Android apps were transformed to a unique IR using WALA [58]–a program analysis framework.

Meanwhile, 22 studies [e.g., S6, S8, S10, S13, S14] detected misuse by executing applications and analyzing their runtime, eliminating the need for accessing source or binary code. However, this method is resource-intensive and time-consuming due to the deployment and execution phases. Figure 10(b) illustrates the distribution of testing input types across the reviewed studies. Notably, several studies employed multiple input types for analysis, which were categorized under all relevant types. For instance, some studies [e.g., S19, S24] examined potential misuses by analyzing binary code and then detected exploitable misuses at runtime by executing the applications.

6.3 Output Types

Common ways to support developers in addressing misuses are issuing ***misuse alarms*** (69 studies), performing ***source attribution*** (4 studies), generating ***security advice*** (4 studies), and assisting with ***fixing*** (4 studies) (Figure 10(c)). In the following, we elaborate on output types other than misuse alarms, which were exclusively explored in the context of crypto and SSL/TLS APIs by the literature.

Source attribution: The objective of source attribution is to determine whether a misuse originates from an application code or a third-party library. It is beneficial for developers to identify libraries with misuses and avoid using them, and for researchers to avoid over-counting misuses by identifying those that stem from libraries [S12]. Three studies investigated the primary source of crypto and SSL/TLS misuse and, interestingly, showed that third-party libraries are the major reason for misuse in Android applications [S1, S12] and popular Python projects in GitHub [S20]. Another study [S35] evaluated a large database of third-party libraries and found that crypto misuses are very common among widely used advertising libraries. It further identified affected Android apps through third-party library detection.

Security advice: Developers often face challenges while trying to fix identified issues, leading to new mistakes [59]. Integrating security advice with crypto APIs has been shown to significantly improve the accuracy and security of code [S59]. Tools like FixDroid [S5] offer real-time feedback and suggestions for quick fixes within the development environment (e.g., Android Studio). Additionally, studies have explored educational approaches to help students learn the proper use of crypto APIs [S63, S17].

Repair: Studies exploring security advice and fix suggestions often lack the ability to provide customized fixes for a given vulnerable program. Only 4 studies [S2, S16, S17, S46] proposed methods for generating such fixes for crypto and SSL/TLS misuses. One simple approach involves manually crafting patch templates, as performed by CDRep [S16] and CryptoTutor [S17]. However, these templates have limited coverage for diverse misuse variations. In contrast, Seader [S2] and VuRLE [S46] automatically generated customized fixes from code examples. They compared ASTs of a given vulnerable code and its corresponding repaired code to extract the edit operations

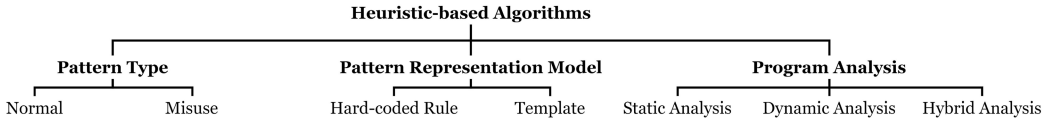


Fig. 11. Taxonomy of Heuristic-based algorithms for detecting misuse of security APIs.

necessary to transform a vulnerable program into a secure one. Using edit operations, Seader generated an abstract fix for a vulnerable program, including abstract variables that were replaced by concrete variables to customize fixes. On the other hand, VuRLE customized the edit patterns for a specific vulnerable program and applied the customized changes to repair vulnerabilities.

6.4 Automation Mode

The analysis techniques reviewed can be categorized by their automation level: ***manual*** (6 studies), ***semi-automated*** (57 studies), or ***automated*** (6 studies) (Figure 10(d)). Most studies fall under the semi-automated approach, relying on predefined patterns for correct and incorrect API usage (details in Section 6.5). Automated approaches learn detection models from code examples, eliminating manual effort but requiring labeled datasets for training. Manual analysis involves human inspection of real-world code (e.g., GitHub projects [S61] or forum posts [S64, S66]) to identify misuses.

6.5 Analysis Algorithms

We classified the reviewed analysis algorithms into two main categories: ***heuristic-based*** (66 studies) and ***ML-based*** (3 studies). This section explores the methodologies within each group.

6.5.1 Heuristic-based. These algorithms typically involve inferring patterns that represent correct or incorrect usage of security APIs and then applying program analysis techniques to identify whether the application being tested matches these patterns. Figure 11 shows the taxonomy of heuristic-based approaches based on the *adopted pattern types, pattern representation models*, and *program analysis techniques*, which are elaborated on below.

A. Pattern Type: API misuses can be detected using two types of patterns: ***normal*** (12 studies) or ***misuse*** (54 studies). Misuse detection through normal patterns involves modeling correct API use and identifying any deviations from it as misuse. While APIs can be misused in numerous ways, only a small subset corresponds to proper usage. Thus, this approach can identify a wide range of misuse types through limited normal patterns. However, this approach suffers from producing high false alarms when it fails to model normal patterns thoroughly. An alternative solution is to model incorrect API uses and identify any matches with them as misuse. However, predicting all possible ways that a developer could misuse an API is a challenging task, so this approach may not capture all misuses. Nevertheless, most of the studies in our review rely on misuse patterns to avoid high false alarms in approaches based on normal patterns.

B. Pattern Representation Model: Two main approaches for modeling patterns are ***hard-coded rules*** and ***templates***. The former involves hard-coding a fixed set of rules into the misuse detection algorithm, against which applications are evaluated. For instance, *"Don't use a constant key for encryption"* or *"Don't store access tokens on clients"* are example rules used to detect crypto and OAuth misuses, respectively. Most studies relied on hard-coded misuse patterns to detect misuse of crypto (31 studies), SSL/TLS (21 studies), OAuth (5 studies), Fingerprint (1 study), and SafetyNet Attestation (1 study) APIs (Figure 12(a)). CryptoLint [S4], for instance, hard-coded 6 misuse patterns for detecting crypto misuses, which were later used and expanded in other studies [e.g., S12,

S16, S29, S35]. Furthermore, one study [S54] used hard-coded normal patterns to investigate the compliance of Android apps with the best current OAuth practices.

Approaches based on hard-coded rules are straightforward to design and implement but have limitations. Notably, they are restricted to detecting only a limited set of misuses, making it difficult to extend beyond predefined rules. The ever-evolving threat landscape of security APIs requires developing more adaptable methods for pattern representation models. With this goal, some researchers proposed using templates to abstractly represent secure or insecure use of security APIs that include ***language-based, graph-based, finite state machine***, and ***code-based*** templates.

(i) Language-based templates rely on a syntax-based representation of patterns. CrySL [S11], for instance, is a language designed for crypto experts to specify the normal use of crypto APIs. Several studies [S3, S11, S36, S40, S58, S61] used CrySL to detect crypto misuses. Meta-CrySL [S55] is an extension of CrySL that helps manage variations in the API and security standards specified within CrySL. Another study [S56] introduced a formal model for security annotations that describe properties ensuring the secure usage of the WebCrypto API. Furthermore, an anti-protocol language was introduced to describe common misuse patterns for the OAuth API [S30].

(ii) Graph-based templates represent key elements used while interacting with APIs as nodes and the correlations between these elements as edges. SSLint [S9], for instance, models the proper use of the SSL/TLS API based on the program dependency graph representing critical API call sites, variables, parameters, and conditions.

(iii) Finite State Machine (FSM) templates represent an application's behavior while using an API through a finite number of states and transitions between them. For example, two studies [S51, S53] modeled the regular operation of OAuth using FSMs, where sending an HTTP(S) request or receiving an HTTP(S) response triggers the transition between states. FSMs were also used to model misuse patterns of SSL/TLS [S25] and Spring [S38] APIs. The research in [S38] implemented FSMs to monitor the program's authorization state for each type of misuse. Transitions between states occur when method calls are made to authorize the user or gain access to a critical resource.

(iv) Code-based templates: Crafting language-, graph-, and FSM-based templates requires substantial domain knowledge and manual effort to specify the critical elements for API usage, their correlation, and modeling them as templates. An alternative solution is to automatically infer templates from sample security API use cases within the source code. In our review, one study [S2] used (insecure, secure) code pairs from prior research and compared their ASTs to identify edit operations. Vulnerability and repair templates were then generated based on a data-dependency analysis of ASTs and variable abstraction. The vulnerability template was used to detect misuse through pattern matching, while the repair template was used to generate customized fixes. Another study [S46] analyzed commits from 48 GitHub applications to manually identify misuses. ASTs were then used to identify required edit operations for fixes, which were clustered later based on their similarities. Finally, each cluster was generalized to vulnerability and repair templates to be used for detecting and fixing misuses. While code templates enable automatic pattern generation, they are inherently limited to known misuses reflected in existing code.

Table 5 summarizes descriptions, strengths, and weaknesses of pattern types and representation models, and Figure 12(a) shows their distribution in primary studies.

C. Program Analysis Techniques: Our review identified three categories for dividing program analysis techniques based on their reliance on code execution: ***(i) static analysis***, ***(ii) dynamic analysis***, and ***(iii) hybrid analysis***. In the following, we investigate these methods and their adoption in the existing literature.

(i) Static Analysis: Static analysis involves examining (recovered) source code, binary code, or an intermediate representation of binary code without executing the application. This approach

Table 5. Pattern Inference Categorizations with their Descriptions, Strengths, and Weaknesses

Type	Description	Strengths	Weaknesses
		Pattern Type	
Normal	Patterns are inferred from correct uses of APIs, and any violation of these patterns is considered as misuse.	• Limited number of patterns • Incomplete specification does not result in missed vulnerabilities	• Susceptible to high false alarm rates
Misuse	Patterns are inferred from incorrect uses of APIs, and any matches with these patterns are considered as misuses.	• Incomplete specification does not result in false alarms	• Difficult to capture all possible patterns • Incomplete specification results in misuses being missed
		Pattern Representation Model	
Hard-coded rules	Patterns are defined as a set of rules.	• Simple to design	• Dependent on domain knowledge • Hard to extend to new misuses
Template	Patterns are abstracted via a higher-level template.	• Easier to extend to new misuses	• Difficult to design a template from instances

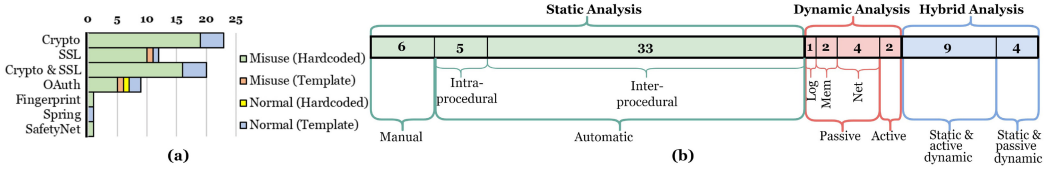


Fig. 12. (a) Distribution of pattern types and representation models used by primary studies for each security API (b) Number of primary studies over program analysis technique in heuristic-based approaches for misuse detection.

is also known as *white-box* testing as it requires application code or its implementation details to identify misuses. Static analysis offers significant advantages: it is resource- and time-efficient, and it can achieve high code coverage. Additionally, developers can integrate static analysis tools into their daily workflow to identify misuse early in the development process. However, static analysis also has limitations, such as high false alarm rates caused by infeasible misuses (that never occur at runtime) and failure to capture runtime misuses.

The main objective of static analysis is to determine the possible values of the parameter objects in a relevant API call and examine them against normal or misuse patterns to detect misuse. This is achieved through ***data flow analysis***, which typically uses *program dependency graphs* or *abstract syntax trees* to understand how data is used and manipulated within a program. There are two types of data flow analysis: ***intra-procedural*** and ***inter-procedural***, depending on whether the interactions between different procedures or functions are considered or not. Most static approaches in our review rely on inter-procedural analysis, which enables the capture of more complex misuses. OAuthLint [S30], for instance, performed inter-procedural data flow analysis using Flowdroid [60] to identify key elements for misuse patterns. Another tool based on inter-procedural analysis is $\text{CogniCrypt}_{\text{SAST}}$ [S11], which was designed as a compiler for CrySL (a language-based template for normal use of crypto APIs; detailed in pattern representation models) to check Java source code for compliance with CrySL and generate code for common crypto tasks. Several studies [e.g., S36, S40, S58, S61] used $\text{CogniCrypt}_{\text{SAST}}$ to detect misuse of crypto APIs. There are also several studies [e.g., S28, S38, S59] based on intra-procedural analysis. For example, one study [S28] used data flow within cryptographic functions to identify paths taken by a parameter from its initial origin to its ultimate use within a function.

To achieve more efficient misuse detection, several studies applied ***program slicing***, which simplifies the complexity of a program by removing parts of the code that are irrelevant to a specific analysis or task [61]. This is accomplished by computing a set of program statements that affect (backward slicing) or are affected by (forward slicing) a given slicing criterion, which is typically an API parameter, based on data flow [S1]. For instance, CryptoTutor [S17] applied inter-procedural data flow analysis and program slicing to detect crypto misuses in Java code. Similarly,

CryptoLint [S4] used inter-procedural backward slicing to track flows between crypto parameters and operations. Later, BinSight [S12] and CDRep [S16] leveraged CryptoLint to examine the current state of crypto API usage in Android applications, with additional efforts toward source attribution [S12] and repair [S16]. Amandroid [S44] also applied inter-procedural data flow analysis to assess the security state of Android apps in terms of data leaks, data injection, and improper use of crypto and SSL/TLS APIs. Program slicing was also used in the context of the Fingerprint API to classify applications into different security levels based on the state of their API use. This involved performing inter-procedural backward slicing to extract API parameters as features in a rule-based classification system [S37].

Although program slicing improves the efficiency of static analysis, it may lead to large memory and runtime overhead on massive-sized projects. To address this challenge, CryptoGuard [S1] proposed a tradeoff between accuracy and scalability by performing on-demand slicing. This approach limits the analysis to methods that have the potential for security impact, effectively reducing the size of the code that needs to be analyzed. Additionally, it utilized refinement algorithms to remove irrelevant language-specific elements and mitigate the high rate of false alarms in static analysis. Later, another study [S39] used CryptoGuard to detect misuse of crypto and SSL/TLS APIs in Android applications.

Another technique to minimize false alarms in static analysis is ***symbolic execution*** [62] that executes a program by using symbolic values as inputs, rather than concrete values, and expressing the values of program variables as symbolic expressions of these inputs. Several studies, such as SSLDoc [S21] and TaintCrypt [S25], leveraged symbolic execution to statically detect security API misuse by creating program path traces that capture semantic information for each targeted API. Another study [S18] performed a simple variant of symbolic execution to extract crypto API sequences from Android applications, which were then used to learn probabilistic models to predict misuses. Additionally, some studies performed manual code analysis to detect crypto misuses [S35, S60, S61, S64, S66] and OAuth misuses [S48].

(ii) Dynamic Analysis: Dynamic analysis involves executing the code of an application and monitoring its behavior during runtime. It is also known as *black-box* testing as it treats applications as black boxes and only considers the external behavior of an application at runtime. Dynamic analysis approaches do not usually produce false positives and can capture misuses occurring during runtime. However, dynamic analysis is resource-intensive and time-consuming, involving tasks such as installing, configuring, and testing, some of which may require human intervention [S23]. It also has limitations in terms of code coverage. There are two types of dynamic analysis for discovering software vulnerabilities, including API misuses: ***active*** and ***passive***. *Active dynamic analysis* involves intentionally attempting to exploit vulnerabilities or cause disruptions in a system. In contrast, *passive dynamic analysis* focuses on collecting data and observing behavior without trying to cause harm.

The passive dynamic analysis examines execution logs, the memory state of a program, or network traces to gain insight into its behavior. Considering the observed data, we have categorized passive dynamic approaches into ***log***, ***memory***, and ***network analysis***. *Log analysis* involves collecting runtime information and execution traces and performing offline analysis after the execution is completed. While the offline analysis does not affect the application's performance [S10], it can generate large log files, creating an I/O bottleneck slowdown [S13]. In our review, a few studies performed log analysis to detect misuse of security APIs. For example, one study [S10] examined logs that record parameters relevant to crypto API calls to find matches with some misuse patterns. *Memory analysis* was also utilized by some studies for misuse detection. For example, K-Hunt [S13] tracks memory buffers that store encryption keys to verify whether keys are

Table 6. Program Analysis Techniques with their Descriptions, Strengths, and Weaknesses

Type	Description	Strengths	Weaknesses
Static	Static analysis examines the application's code against API usage constraints.	• Doesn't require program execution and is scalable to a large number of applications.	• Applicable only to open-source projects • Suffers from high false alarm rate
Dynamic	Dynamic testing executes the software and validates output or runtime information against API usage constraints.	• Able to capture misuses occurring during runtime	• Requires code execution • Costly and not scalable to a large number of projects

generated and transmitted securely. It started with a lightweight dynamic analysis to gather runtime information required to locate memory buffers where crypto keys were stored. Meanwhile, several studies adopted *network analysis* to detect misuses of APIs such as SSL/TLS and OAuth. One study [S52] evaluated the implementation of CSRF protection in OAuth transactions by checking the presence or absence of a *state* variable in URLs.

In addition, some studies performed active dynamic analysis to verify the results obtained from network analysis. Active dynamic analysis can include a range of techniques, such as ***penetration testing***, which simulates a real-world attack on a running application to identify any misuses that could be exploited. It is considered the most effective approach to uncover exploitable misuses and avoid false alarms. One study [S50] manually analyzed the HTTP messages to capture the information flow of SSO credentials and detect potential misuse of OAuth. It further designed exploits to prevent manual inspection errors. Similarly, another study [S49] performed network analysis, followed by examining the feasibility of a CSRF attack to uncover exploitable misuses of OAuth.

(iii) Hybrid Analysis: Attempts have been made to combine static and dynamic analyses in a ***hybrid*** approach to leverage the strengths of both techniques and overcome their weaknesses. Table 6 provides a concise summary of the strengths and weaknesses associated with static and dynamic analysis techniques. To mitigate the risk of false positives in the static analysis, some researchers proposed a hybrid approach that typically applies static analysis to identify potential misuses, followed by dynamic analysis to validate the results. Several studies [S8, S14, S19, S24, S57, S67] evaluated Android apps against a MitM attack to verify misuses reported by static analysis. Another study [S23] applied manual static analysis to find potential crypto misuses in Android apps and then performed dynamic memory analysis to examine the crypto libraries invoked during execution. This approach enables the detection of misuses that are feasible at runtime. Another study [S69] used a combination of static and dynamic analyses to find Android applications that call the SafetyNet Attestation API during their execution. Next, it did a manual static analysis to find vulnerable applications with potential misuses, followed by bypassing the SafetyNet Attestation checks to confirm the misuses.

Static analysis can also serve as a guide for dynamic analysis, reducing the time and memory consumption of dynamic analysis by pruning its exploration space. Some studies [e.g., S19, S24, S67] employed a preliminary static analysis to detect misuses. They further used static analysis for method call graphs to identify the entry points that trigger the execution of vulnerable methods. These entry points were then used to generate inputs for running applications during dynamic analysis, resulting in a more efficient analysis with a reduced input space. Another study [S6] combined static and dynamic analysis techniques to detect crypto misuses in iOS apps. It first used static analysis to find the locations of crypto APIs and then monitored those API calls at runtime using *API hooking* techniques. Misuses were detected by analyzing the execution logs, which record parameter values and other relevant information. AuthDroid [S47] also adopted a hybrid approach to detect OAuth misuse in Android apps. It uses static analysis to extract the basic elements of OAuth (e.g., user-agent, the identity of SP) from the app, then uses a MitM proxy in dynamic analysis to find API misuses in the authentication process. While the mentioned studies followed a static-dynamic approach in detecting misuses, one study [S26] has taken a different

hybrid approach by first simulating a MitM attack to find vulnerable apps, and then manually performing static code analysis to identify the root causes of misuses.

6.5.2 ML-based. Our review found only 3 studies using ML-based algorithms to detect security API misuse. The main idea is to classify API usage instances within a given application as correct or incorrect using features reflecting the application's behavior. Below, we examine the feature engineering and classification components of these approaches.

A. Feature Engineering: Three types of features were identified in the existing literature for building security API misuse detection models, which are ***sequential, word-***, and ***graph-based*** features.

(i) Sequential features: One study [S18] used API sequences representing both API orders and API arguments to learn probabilistic models for misuse detection. To this end, they used static analysis to extract possible traces for each reachable method from application binary files. Furthermore, they performed a simple variant of symbolic execution on each trace and then filtered traces of irrelevant APIs.

(ii) Word-based features: **Term frequency-inverse document frequency** (**tf-idf**) is a common technique used in **Natural Language Processing** (**NLP**) to evaluate the importance of a term in a document or corpus. Recent advances in NLP have inspired many researchers to apply it to analyzing source code by considering it as natural-language text. In our review, one study [S45] extracted tf-idf from source codes to train a misuse detection model.

(iii) Graph-based features: One study [S65] utilized graph-based features to analyze the usage of security APIs in source code. First, the source code was parsed to an AST and then modeled through graph embedding techniques, **Bag of Graphs** (**BoG**), and node2vec. These techniques are similar to word embedding techniques in NLP, where words are mapped to vectors based on how often they appear together in text. Similarly, BoG generates a collection of graph bag items representing elements or sub-graphs within a graph. These items are then used to construct a vector representation that captures the local attributes and relationships of the original graph. Node2vec is another graph embedding technique that extracts features from graphs, utilizing a flexible neighborhood sampling strategy.

B. Classification: In one study [S18], two probabilistic models, **Hidden Markov Model** (**HMM**) and n-gram, were trained using both secure and insecure API sequences. Notably, the labels for these sequences were generated using an existing tool, $\text{CogniCrypt}_{\text{SAST}}$ [S11], designed for detecting misuses of cryptography APIs. The trained models were employed to predict the probability of a given API sequence being secure. An API sequence was considered insecure if its probability fell below a pre-defined threshold. The study also addressed the problem of identifying misuse locations within an insecure sequence by using a distance measure based on the probability of an API misuse at possible locations. In another study [S45], code snippets with the usage of security APIs were mined from SO and then classified using a **Support Vector Machine** (**SVM**) model. A small set of extracted code snippets was manually labeled to build the training dataset. Similarly, the approach proposed in [S65] used SVM, but trained a separate classifier for each misuse type, enabling the model to identify both the presence and type of API misuse. The classifiers were trained using labeled data containing correct and incorrect API use instances from two existing benchmarks for evaluating crypto misuse detection techniques, namely datasets by Braga et al. [63] and Fischer et al. [37].

7 RQ4: Evaluation Methods

Our review identified 25 studies that performed experiments to assess the performance of misuse detection techniques. Out of these, 5 studies [S31, S32, S34, S41, S42] evaluated the performance

of various static analysis tools in detecting crypto and SSL/TLS misuses. Following, we discuss various metrics, benchmarks, and strategies adopted for evaluation.

7.1 Evaluation Strategies

In our review, various techniques were used to evaluate the performance of misuse detection models. We noticed only seven studies designed experiments using ***public benchmarks*** (Section 7.2). Existing benchmarks are typically limited in terms of scale and diversity of test cases. To address this issue, one study [S34] explored the ***automatic generation of test cases*** using mutation operations. It generated over 20,000 test cases, which were used to evaluate several crypto misuse detection tools and identify their flaws, such as failure to detect insecure algorithms provided in lowercase. In addition, 19 primary studies conducted ***case studies*** or ***manually analyzed*** a subset of their dataset or a subset of reported misuses to verify their results. For instance, one study [S10] randomly selected 150 Android apps out of 1,780 analyzed apps to validate their findings, and another study [S58] randomly sampled 157 misuses and manually verified them to gain a deeper understanding of common false positives.

The study [S45] created a dataset for 5-fold cross-validation by manually labeling a collection of security-related code snippets from SO as either secure or insecure that were used for evaluating its proposed ML-based algorithms. Unlike the study [S45], the study [S18] relied on an ***existing tool***, CogniCrypt$_{SAST}$ [S11], to label crypto API use cases in Android applications and provide a labeled dataset for training, validating, and testing purposes of its ML algorithms, which makes the results dependent on the performance of the employed tool.

Another evaluation technique, adopted by 3 studies, involves ***executing attacks*** to validate the results and identify exploitable misuses. For instance, one study [S6] executed two ethical attacks on two applications and successfully retrieved personal information encrypted and transmitted over the network. Meanwhile, 16 studies disclosed misuses identified in real-world projects, some of which analyzed the feedback they received from developers. This analysis provided valuable insights into developers' requirements for misuse detection tools, disregarded in existing approaches.

Lastly, we found 3 studies that conducted ***user studies*** to evaluate the usability of tools with warning messages and suggestions for fixes. These studies involved 39 developers [S5], 8 developers [S42], and 53 developers [S59] and showed that security advice could improve the usage of crypto APIs in users' codes. More importantly, they highlighted the need for detailed and specific solutions that are comprehensible and feasible for developers.

7.2 Evaluation Benchmarks

Benchmarks are critical for evaluating detection techniques and identifying their strengths and weaknesses. Unfortunately, there is an inadequate number of publicly available benchmarks, and all of them are limited to test cases for some misuses of Java crypto and SSL/TLS APIs. Table 7 lists 9 public benchmarks commonly used by researchers. Among them, the first five benchmarks were specifically created to evaluate and compare the performance of misuse detection approaches for security APIs.

CryptoAPI-Bench includes synthetic source code examples with crypto API misuses, false positive tests, and correct API uses. It offers both basic test cases and advanced test cases that involve more complex scenarios. CryptoAPI-Bench was designed to assess static tools. ***CryptoAPI-Bench**** [S10] is an extension of CryptoAPI-Bench with additional cases suitable for evaluating dynamic approaches. CryptoAPI-Bench is not suitable for evaluating the scalability of a tool, as all test cases are lightweight by design. To address this limitation, another study [S32] created ***ApacheCryptoAPI-Bench*** using 10 real-world Apache projects that are complex programs with

Table 7. Public Benchmarks for Evaluating Crypto Misuse Detection Techniques

Benchmark	Size	R/S[1]	Description	Ref
CryptoAPI-Bench (2019) [S1][64]	181 test cases	S	Benchmark for evaluating crypto misuse detectors containing 45 basic and 136 complex test cases with crypto API misuses, false positive tests, and correct API uses	S1, S2, S31, S32
CryptoAPI-Bench* (2021) [S10]	198 test cases	S	CryptoAPI-Bench with further cases suitable for assessing dynamic approaches, totally consisting of 157 crypto misuse cases, and 41 normal test cases	S10
ApacheCryptoAPI-Bench (2020) [S32][65]	120 test cases	R	Ten real-world Apache projects including 79 basic test cases and 42 advanced test cases, suitable for assessing the scalability of misuse detection approaches	S32
Braga et al.'s dataset (2017) [63]	384 test cases	S	Contains 202 misuses (positive cases) and 182 normal uses (negative cases) for Java Cryptography Architecture	S41, S65
Fischer et al.'s dataset (2019) [37]	16,346 test cases	R	6,246 secure cases and 10,100 insecure cases for the use of crypto API adopted from code snippets available at SO posts	S65
MUBench (2016) [66][67]	21 apps	R	Benchmark for evaluating API-misuse detectors containing instances of crypto API misuses from 62 Java programs	S31
OWASP (2021) [68]	975 programs	R	Java test suite designed for evaluating vulnerability detectors, containing 477 programs with labeled misuses of security APIs and 498 programs with correct uses	S31
DroidBench (2015) [69]	21 apps	R	Benchmark apps for evaluating the performance of static information-flow analysis of Android apps, including crypto misuse test cases	S44
ICC-Bench (2017) [70]	24 apps	R	Benchmark apps for evaluating the performance of static analysis to detect inter-component data leakage problem of Android apps, including crypto misuse test cases	S44

[1]Real or Synthetic

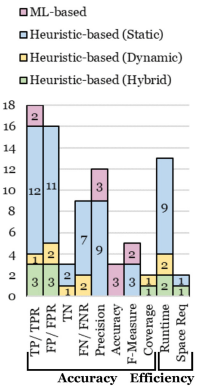


Fig. 13. Evaluation metrics Distribution categorized by detection techniques.

numerous and lengthy code files. This benchmark is therefore appropriate for assessing the scalability and applicability of existing approaches to real-world applications.

Two additional benchmarks for evaluating crypto misuse detection techniques are ***Braga et al.'s*** [63] and ***Fischer et al.'s*** [37] datasets that contain labeled instances of secure and insecure use of the Java Cryptography Architecture API. Braga et al.'s dataset consists of synthetic Java source codes, while the latter consists of real-world code snippets collected from SO. Both datasets were used by the study [S65] to train and test an ML-based detection technique. The last four benchmarks have been designed to evaluate API misuse or vulnerability detectors and include some test cases for evaluating crypto misuse detection techniques as well.

7.3 Evaluation Metrics

We have identified 10 evaluation metrics that are commonly used to measure the performance of security API misuse detection techniques. These metrics are grouped into two categories: ***detection effectiveness*** and ***computation efficiency***. Metrics for detection effectiveness are typically calculated using ***True Positive (TP)***, ***False Positive (FP)***, ***True Negative (TN)***, or ***False Negative (FN)*** values. While detecting all misuses is crucial, having a high number of false alarms can be highly time-consuming and burdensome for developers. Hence, the primary objective of misuse detection is to maximize the ***True Positive Rate (TPR)*** or ***Recall (R)***, while minimizing the ***False Positive Rate (FPR)***. These two metrics are the most commonly used. Other metrics, such as ***True Negative Rate (TNR)***, ***False Negative Rate (FNR)***, ***Precision (P)***, ***Classification Accuracy***, ***F-Measure***, and ***Coverage*** have also been used by researchers to measure the effectiveness of misuse detection.

Several primary studies also considered the computation efficiency of detection techniques, which was measured using the ***runtime*** and ***space*** complexity required for misuse detection. Evaluating computation efficiency is crucial in demonstrating the suitability of these techniques for real-world applications. Classification accuracy was used only by ML-based detection techniques, and coverage measurement is exclusive to dynamic analysis, as it measures the proportion of the program code that has been executed during testing. Figure 13 illustrates the distribution of the identified evaluation metrics that are categorized by detection technique.

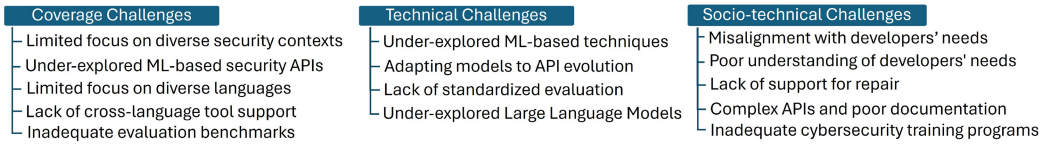


Fig. 14. Open issues in enhancing and evaluating security API use.

8 Open Research Challenges and Future Directions

Our review identified 14 critical open issues in enhancing and evaluating security API use, categorized as coverage, technical, and socio-technical issues (detailed in Figure 14). In the following, we discuss these challenges and propose recommendations for future research to address them.

8.1 Coverage Challenges

This category includes challenges arising from the limited scope of existing research as discussed below.

8.1.1 Limited Focus on Diverse Security Contexts. Although the focus of primary studies on crypto and SSL APIs (83% of reviewed studies) highlights their crucial role in secure software development, it raises a warning alarm about potential vulnerabilities caused by misusing security APIs that receive inadequate or even no attention from the research community. Meng et al.'s analysis of SO posts [71] revealed that Spring Security is the most popular option among Java developers in secure coding practices. However, our review found only one study dedicated to this overly complicated and poorly documented API. However, research suggests that misusing other security APIs, including those designed to store and access sensitive information, can have significant security implications [72–74]. As a result, developing effective mechanisms to detect and prevent API misuse in these contexts is critical.

8.1.2 Under-explored ML-based Security APIs. Novel security APIs have been developed to facilitate the integration of applications with cutting-edge security technologies, including various ML-based authentication schemes such as facial recognition, vocal recognition, and iris-based authentication [75]. While their adoption is increasing in various applications, research on their usability and usage patterns remains limited. The reliance of these APIs on probabilistic ML models introduces unique challenges in detecting their misuse [76–78]. Unlike traditional APIs, where misuse might lead to explicit errors, misuse of ML-based APIs can result in suboptimal or incorrect predictions that deviate from human judgment [77]. This makes the analysis and identification of misuse significantly more complex. Therefore, further research is imperative to devise and develop methodologies to identify and mitigate the misuse of ML-based security APIs.

8.1.3 Limited Focus on Diverse Programming Languages. Over 70% of the studies reviewed focused on Java, reflecting its extensive adoption and complex API design. Nonetheless, it is essential to recognize the significance of exploring and addressing existing challenges for using APIs in other widely used programming languages such as Python and C. Several primary studies on non-Java languages indicate that developers in other languages are also likely to make mistakes when using security APIs [e.g., S6, S9, S13]. This underscores the critical need for developing tools across diverse programming languages, with essential support from the research community. It is highly recommended that researchers broaden their research scope to include non-Java languages in their analysis of security API misuse.

8.1.4 Lack of Cross-language Tool Support. Along with the need to develop tools for various languages, there is a need to develop cross-language tools. A study [S20] analyzing MicroPython projects suggests that developers working in the embedded domain often use C language to implement cryptography operations. This indicates the need for tools with cross-language analysis capability to track program information across multiple programming languages. Furthermore, Meng et al.'s analysis of SO posts [71] has shed light on the challenge of cross-language data handling of cryptography APIs, where developers struggle to encrypt data in one language (e.g., PHP or Python) and decrypt data in another language (e.g., Java). To address these issues, future research should address the development of cross-language approaches to ensure secure software development across different programming languages.

8.1.5 Limited Availability and Scope of Evaluation Benchmarks. Benchmarks are crucial to compare and evaluate the performance of proposed approaches and tools for security API misuse detection and to identify the areas for further improvement. However, our review identified only 9 publicly available benchmarks, out of which only 5 were specially designed for security API misuses. All these benchmarks are limited to test cases for some misuses of Java crypto or SSL/TLS APIs. Notably, three out of these five benchmarks are synthetic, rendering them ineffective for evaluation in real-world settings. Thus, there is a desperate need to expand the scope of benchmarks to cover a broader range of security APIs, real-world misuses, programming languages, and software platforms suitable for evaluating tools developed for diverse security APIs in real-world scenarios. On the other hand, existing benchmarks are mostly limited to test cases for evaluating static tools, which renders them unsuitable for assessing the performance of dynamic analysis tools [S10]. Therefore, benchmarks need to come with test cases for evaluating dynamic analysis tools. It is also crucial to continuously update existing benchmarks to incorporate new misuses and the constant evolution of APIs. Furthermore, we recommend versioning benchmarks to address issues such as concept and temporal drift.

8.2 Technical Challenges

Our review identified five technical-related open issues, detailed below.

8.2.1 Limited Research on the Application of ML-based Techniques. Our SLR identified a critical gap: *a scarcity of state-of-the-art ML-based models* for identifying security API misuse. Over 95% of the reviewed studies rely on traditional heuristic-based approaches that require significant domain knowledge and result in labor-intensive, time-consuming, and error-prone processes. Manual analysis of identified misuses in one study [S58], using heuristic-based CrySL [S11], also revealed several false positives due to incorrect specifications defined in CrySL. Additionally, these methods often rely on hard-coded rules, making them difficult to adapt to emerging misuses to keep pace with the rapidly evolving security threat landscape. The main obstacle to adopting ML models in this domain is the scarcity of large-scale, labeled datasets. While some studies have manually analyzed and verified identified misuses, none have publicly shared the results of their analysis. We strongly encourage researchers to openly share their findings on security API usage analysis. This would not only support validation and future research but would also significantly benefit the development of large-scale labeled datasets, crucial for both training ML models and establishing evaluation benchmarks.

8.2.2 Under-explored Application of Large Language Models. In recent years, the rise of **Large Language Models** (**LLMs**) has significantly influenced the field of software engineering [79]. Their capability to comprehend and generate code has demonstrated remarkable performance across various tasks like code completion [80], explanation [81], and repair [82]. This suggests

LLMs as potential tools for addressing security API misuse, a critical yet under-explored area. While a study by Wei et al. [78] using ChatGPT demonstrated promising results in detecting and fixing API misuse, its scope is limited to a specific set of ML APIs. The research by Mousavi et al. [83] revealed limitations in LLMs' ability to generate secure code for security APIs. Further research is crucial to gain an in-depth understanding of LLMs' strengths and limitations in addressing security API misuse. Additionally, future investigations should explore avenues to enhance security awareness in LLMs, including strategies such as security-enhanced training, integrating security considerations during fine-tuning or prompt tuning, and adapting LLM generation to reinforce adherence to established security best practices.

8.2.3 Challenges in Adapting Models to API Evolution. The ever-changing nature of APIs presents a significant challenge for misuse detection models. As APIs adapt to new requirements and security concerns, detection models must constantly keep pace to identify new misuse patterns arising from these changes. However, the rapid evolution often outpaces existing models' ability to detect new forms of misuse. In our analysis, we noted the deprecation of two APIs, Fingerprint and SafetyNet Attestation, yet no solutions have addressed adapting misuse detection models to their alternatives, Biometrics [84] and Play Integrity [85] APIs. Zhong and Meng's recent work [86] takes a step toward addressing this issue by introducing a compiler-directed approach that automatically updates call sites when compilation errors indicate that a previously valid API usage has become incompatible. This offers a promising direction for bridging the gap between evolving APIs and misuse detection models when changes result in compilation failures. However, many evolution scenarios–particularly in the security domain–do not manifest as compilation errors. Future research should therefore extend automated migration techniques to handle such cases and integrate these techniques with misuse detection models to ensure their continued effectiveness as APIs evolve. Another significant issue is the overreliance on hard-coded rules (over 95% of studies), which limits the ability to detect emerging misuse patterns. We therefore recommend that future work investigate ML-based models and efficient incremental learning algorithms to address concept drift as APIs continue to evolve.

8.2.4 Lack of a Standardized Evaluation Framework. Pendlebury et al. [87] identified two critical sources of bias that inflate experimental results in ML-based malware classification: spatial bias and temporal bias. Spatial bias stems from unrealistic assumptions about the goodware-to-malware ratio within training and testing data. Temporal bias occurs due to unrealistic time splits within datasets, where future knowledge is included in the training data. Their proposed evaluation framework addresses these issues by incorporating spatial and temporal constraints. This concern extends beyond malware classification to any machine learning model in the security domain that deals with constantly evolving threats and time-sensitive data, including ML-based security API misuse detection. Furthermore, heuristic-based approaches should also be tested under conditions that reflect actual usage scenarios and the evolution of APIs over time. However, no reviewed studies explicitly considered such biases. To ensure accurate assessment of misuse detection in realistic settings, the research community must prioritize integrating spatial and temporal constraints into experimental design. This specifically requires augmenting datasets with accurate timestamps and obtaining realistic ratios between normal and misuse cases.

8.3 Socio-technical Challenges

Despite being the primary users of APIs and security tools, developers are often overlooked in both API and tool development. This section discusses five key socio-technical challenges in security API misuse detection.

8.3.1 Inadequate Models to Meet Developers' Expectations. Sixteen studies disclosed misuses identified in real-world projects, some of which [e.g., S1, S10, S31] reported the feedback received from developers. These studies revealed *inadequacies of current tools in meeting the expectations and requirements of developers*, indicating the need for adapting these tools based on user feedback and preferences. In certain cases, developers acknowledged the existence of misuses (71% [S21], 52% [S9], 20% [S29] of reported misuses) and attempted to address them. However, in some cases, they disregarded the identified misuses, believing them to be irrelevant. These misuses were often found in non-sensitive contexts such as security-irrelevant pieces of code, test cases, or archived code [S10, S31]. Additionally, in some cases, developers rejected misuses without concrete exploit demonstrations [S1, S10, S31]. To better align with developers' requirements, practitioners should consider developing detection tools capable of differentiating between security-relevant and security-irrelevant contexts and demonstrating concrete exploit examples for identified misuses. By addressing the human-centric aspects of misuse detection models, tool developers can enhance the usability, effectiveness, and adoption of these tools among developers.

8.3.2 Lack of Understanding of Developers' Needs. Current research on misuse detection models overlooks *understanding developers' needs*. Our review identified a significant gap in this area, with only three studies [S5, S42, S59] evaluating the usability of their tools in assisting developers. To fill this research gap, future research needs to focus on understanding and identifying developers' requirements and expectations from misuse detection models. We recommend conducting user studies to gather insights directly from developers to achieve a more in-depth understanding of their needs. By understanding and analyzing these requirements, researchers can craft human-centric strategies for integrating user feedback into the design and development phases of misuse detection tools.

8.3.3 Lack of Support for Repair. Developers may face challenges while trying to fix identified misuses, potentially introducing new mistakes [S42]. An analysis of vulnerability disclosures, reported by the reviewed studies, reveals instances where developers acknowledged misuses but struggled to resolve them due to several limitations. Operational constraints, like maintaining backward compatibility, can restrict their ability to implement necessary fixes [S1]. Additionally, existing tools often provide inadequate repair guidance, lacking the detailed information required for repair [S31]. Further, the inherent complexity of implementing secure solutions poses a significant challenge for developers [S31]. To address these challenges, misuse detection tools should be complemented with comprehensible, detailed, and actionable fixing suggestions. While some studies offer general repair guidance for specific misuse types [S5, S59, S63], they lack customized fixes for a given vulnerable program. There are only four studies [S2, S16, S17, S46] for the automated generation of customized fixes for crypto misuses. Existing tools, however, are still inadequate in assisting developers with accurately correcting misuses [S31]. Future research should explore methods for providing detailed and customized suggestions and streamlining automated repair solutions.

8.3.4 Inadequate API Usability and Poor Documentation. The intricate designs of security APIs create a significant barrier to understanding and proper implementation, often overwhelming developers with a complex set of programming options, numerous parameters, return values, and their security implications [7]. This complexity is further compounded by poorly configured APIs with insecure defaults, like JCA's default use of the `ECB` mode for `AES` encryption [6]. Another critical concern arises from inadequate or poor documentation that fails to provide explicit examples of proper usage or even includes API misuses, as seen within documentation by some OAuth API providers [15]. This lack of clear guidance forces developers to rely on potentially unreliable sources like forum posts, which can lead to copying and pasting incorrect suggestions

that contain misuse instances [16]. To address these challenges, there should be a focus on user-centric approaches to security API design. Security API development should prioritize clarity and ease of use while ensuring secure configurations by default and providing well-written documentation that streamlines the effective use of security APIs even for novice developers. To achieve this, we recommend that researchers investigate and identify usability issues in existing security APIs and devise effective mitigation strategies.

8.3.5 Lack of Effective Cybersecurity Training Programs. Inadequate cybersecurity training for developers is a major contributor to security API misuse [17]. Meanwhile, the threat landscape for security APIs is continuously evolving, with new attack techniques and vulnerabilities being discovered regularly. Hence, ongoing cybersecurity education and training for developers is crucial. However, research to support effective training programs remains limited [88–90]. Therefore, future research should focus on identifying and overcoming barriers to the adoption of existing developer security training programs. The research also needs to explore innovative technologies, including virtual reality, gamification, and interactive online platforms for delivering engaging and effective security training [91]. Additionally, given the critical role of security APIs in software security, their secure use should be a top priority in training programs. Moreover, organizations must prioritize investments in developer training programs and ensure that their developers receive regular updates on cybersecurity best practices.

9 Threats to Validity

We followed the guidelines outlined in study [19] to design and conduct our SLR. However, there might still be some biases due to the author's expertise and different perspectives. We took necessary steps to minimize the impact of potential biases, including those in the study selection and review process, which are elaborated upon below:

***Search Strategy*:** One of the common threats to the validity of an SLR is the possibility of missing relevant studies. To minimize this risk, we utilized Scopus, which is the most comprehensive search engine and largest indexing system [20, 21], and supplemented it with the two most frequently used digital libraries, IEEE Xplore and ACM Digital Library [22]. We also conducted a series of pilot searches to establish a search string that would retrieve relevant articles already known to us. In addition, we employed both forward and backward snowballing techniques to locate any other relevant articles that might have been missed by the search string.

***Selection process*:** The potential for subjective bias in the selection of studies cannot be ruled out, as it could be influenced by the author's subjective judgment. To address this concern, we carried out a rigorous and well-defined multi-step process (detailed in Section 3.3) with clear inclusion and exclusion criteria. We also established specific guidelines to exclude low-quality articles. At every stage of the selection process, we carefully deliberated and addressed any uncertainties through discussions among all the authors to minimize the risk of selection bias.

***Data Extraction and Synthesis*:** Human errors and author biases during data extraction, analysis, and interpretation can impact the accuracy of the results and findings. To mitigate this issue, a data extraction form was developed and refined to ensure the collection of adequate and consistent information for answering the research questions. We also conducted fortnightly meetings among all the authors to review and verify the synthesis and interpretation of our quantitative and qualitative analysis. Any disagreements were discussed and resolved collaboratively before finalizing our responses to the RQs.

10 Conclusion

Security APIs play a crucial role in secure software development. Prior studies have shown developers often misuse security APIs, leading to costly software vulnerabilities. Thus, misuse detection

for security APIs has gained significant attention from the research community for ensuring software security. However, the existing literature on the topic is dispersed, and a systematic review was necessary to identify the state-of-the-art approaches and highlight areas that require further exploration. This study presents our research effort aimed at systematically reviewing and rigorously analyzing the literature on misuse detection for security APIs. To the best of our knowledge, this SLR is the first attempt to systematically review the literature on this topic. We have provided an organized evidence-based body of knowledge to enrich this domain by identifying security APIs, their potential misuses, detection techniques, and evaluation methods. In conclusion, based on a comprehensive analysis of 69 primary studies, we identified the following key trends in security API misuse detection research:

(1) We identified 6 security APIs examined for misuse detection, namely cryptography primitives (crypto), SSL/TLS, OAuth, Fingerprint, Spring, and SafetyNet Attestation. Most studies focused on crypto and SSL/TLS, highlighting the need to explore this topic for other security APIs.
(2) We identified a total of 30 distinct types of security API misuses. The primary studies mainly focused on analyzing Android apps, and the most commonly reported misuses were using insecure crypto algorithms for crypto APIs, improper certificate validation for SSL APIs, lack or misuse of the *state* parameter for OAuth APIs, and lack or misuse of cryptography for Fingerprint APIs.
(3) We proposed a taxonomy consisting of heuristic-based and ML-based approaches for misuse detection techniques. Most studies relied on heuristic-based approaches, with 42 studies based on static analysis, 9 studies using dynamic analysis, and 13 studies using a hybrid approach. We found only 3 studies using ML to address misuse detection. Our findings suggest the need to explore the application of ML, DL, and NLP techniques in this area.
(4) We identified 11 metrics for evaluation, grouped into accuracy and efficiency categories. We found only five public benchmarks, particularly designed for security API misuse, which are limited to test cases for crypto and SSL/TLS misuses. These findings highlight the need for further research and development of more diverse benchmarks to facilitate the evaluation of misuse detection techniques for security APIs.

Overall, our SLR offers valuable insights for both researchers and practitioners. Researchers can deepen their understanding of existing work and identify areas for their future studies. Practitioners benefit in two ways - firstly, by using the findings to develop more effective tools for detecting security API misuse and secondly, by gaining insights into potential misuses, enabling them to avoid them in their development processes. Ultimately, this review contributes to the broader goal of promoting secure software development.

References

[1] Martin Georgiev, Subodh Iyengar, Suman Jana, Rishita Anubhai, Dan Boneh, and Vitaly Shmatikov. 2012. The most dangerous code in the world: validating SSL certificates in non-browser software. In *Proceedings of the 2012 ACM Conference on Computer and Communications Security*. 38–49.

[2] Manuel Egele, David Brumley, Yanick Fratantonio, and Christopher Kruegel. 2013. An empirical study of cryptographic misuse in android applications. In *Proceedings of the 2013 ACM SIGSAC Conference on Computer and Communications Security*. 73–84.

[3] Sazzadur Rahaman, Ya Xiao, Sharmin Afrose, Fahad Shaon, Ke Tian, Miles Frantz, Murat Kantarcioglu, and Danfeng Yao. 2019. Cryptoguard: High precision detection of cryptographic vulnerabilities in massive-sized java projects. In *Proceedings of the 2019 ACM SIGSAC Conference on Computer and Communications Security*. 2455–2472.

[4] Antonio Bianchi, Yanick Fratantonio, Aravind Machiry, Christopher Kruegel, Giovanni Vigna, Simon Pak Ho Chung, and Wenke Lee. 2018. Broken fingers: On the usage of the fingerprint API in Android. In *Proceedings of the NDSS*.

[5] Tamjid Al Rahat, Yu Feng, and Yuan Tian. 2019. Oauthlint: An empirical study on oauth bugs in android applications. In *Proceedings of the 2019 34th IEEE/ACM International Conference on Automated Software Engineering*. IEEE, 293–304.
[6] Stefan Krüger, Johannes Späth, Karim Ali, Eric Bodden, and Mira Mezini. 2019. CrySL: An extensible approach to validating the correct usage of cryptographic APIs. *IEEE Transactions on Software Engineering* 47, 11 (2019), 2382–2400.
[7] Matthew Green and Matthew Smith. 2016. Developers are not the enemy!: The need for usable security APIs. *IEEE Security and Privacy* 14, 5 (2016), 40–46.
[8] Mohammadreza Hazhirpasand, Mohammad Ghafari, Stefan Krüger, Eric Bodden, and Oscar Nierstrasz. 2019. The impact of developer experience in using Java cryptography. In *Proceedings of the 2019 ACM/IEEE International Symposium on Empirical Software Engineering and Measurement*. IEEE, 1–6.
[9] Maxime Lamothe, Yann-Gaël Guéhéneuc, and Weiyi Shang. 2021. A systematic review of API evolution literature. *ACM Computing Surveys* 54, 8 (2021), 1–36.
[10] Peter Leo Gorski and Luigi Lo Iacono. 2016. Towards the usability evaluation of security APIs. In *Proceedings of the Clarke, Furnell (Eds.): 10th International Symposium on Human Aspects of Information Security and Assurance, Frankfurt, Germany, July 19–21, 2016*. CSCAN, 252–265.
[11] Michael E. Whitman and Herbert J. Mattord. 2013. *Management of Information Security*. Cengage Learning.
[12] Dick Hardt. 2012. The OAuth 2.0 Authorization Framework. RFC Editor. DOI: https://doi.org/10.17487/RFC6749
[13] Mariusz Krzysztofek. 2018. *GDPR: General Data Protection Regulation (EU) 2016/679: Post-reform Personal Data Protection in the European Union*. Kluwer Law International BV.
[14] U.S. Department of Health and Human Services. 2024. HIPAA (Health Insurance Portability and Accountability Act). Retrieved from https://www.hhs.gov/hipaa/for-professionals/index.html. Accessed May 26, 2025.
[15] Ethan Shernan, Henry Carter, Dave Tian, Patrick Traynor, and Kevin Butler. 2015. More guidelines than rules: CSRF vulnerabilities from noncompliant OAuth 2.0 implementations. In *Detection of Intrusions and Malware, and Vulnerability Assessment: 12th International Conference, DIMVA 2015, Milan, Italy, July 9–10, 2015, Proceedings 12*. Springer, 239–260.
[16] Felix Fischer, Konstantin Böttinger, Huang Xiao, Christian Stransky, Yasemin Acar, Michael Backes, and Sascha Fahl. 2017. Stack Overflow considered harmful? The impact of copy&paste on Android application security. In *Proceedings of the 2017 IEEE Symposium on Security and Privacy*. IEEE, 121–136.
[17] Hala Assal and Sonia Chiasson. 2018. Security in the software development lifecycle. In *Proceedings of the SOUPS@ USENIX Security Symposium*. 281–296.
[18] Barbara A. Kitchenham, Tore Dyba, and Magne Jorgensen. 2004. Evidence-based software engineering. In *Proceedings. of the 26th International Conference on Software Engineering*. IEEE, 273–281.
[19] B. Kitchenham and S. Charters. 2007. *Guidelines for Performing Systematic Literature Reviews in Software Engineering*. Technical Report. Technical report, Ver. 2.3 EBSE Technical Report. EBSE.
[20] Triet H. M. Le, Huaming Chen, and M. Ali Babar. 2022. A survey on data-driven software vulnerability assessment and prioritization. *ACM Computing Surveys* 55, 5 (2022), 1–39.
[21] Roland Croft, Yongzheng Xie, and Muhammad Ali Babar. 2022. Data preparation for software vulnerability prediction: A systematic literature review. *IEEE Transactions on Software Engineering* 49, 3 (2022), 1044–1063.
[22] Guanjun Lin, Sheng Wen, Qing-Long Han, Jun Zhang, and Yang Xiang. 2020. Software vulnerability detection using deep neural networks: A survey. *Proc. IEEE* 108, 10 (2020), 1825–1848.
[23] Bushra Sabir, Faheem Ullah, M. Ali Babar, and Raj Gaire. 2021. Machine learning for detecting data exfiltration: A review. *ACM Computing Surveys* 54, 3 (2021), 1–47.
[24] Orvila Sarker, Asangi Jayatilaka, Sherif Haggag, Chelsea Liu, and M. Ali Babar. 2024. A Multi-vocal Literature Review on challenges and critical success factors of phishing education, training and awareness. *J. Syst. Softw.* 208, C (February 2024). DOI: https://doi.org/10.1016/j.jss.2023.111899
[25] Claes Wohlin. 2014. Guidelines for snowballing in systematic literature studies and a replication in software engineering. In *Proceedings of the 18th International Conference on Evaluation and Assessment in Software Engineering*. 1–10.
[26] Zahra Mousavi, Chadni Islam, M. Ali Babar, Alsharif Abuadbba, and Kristen Moore. 2023. Online Appendix of "Detecting Misuses of Security APIs: A Systematic Review." Retrieved from https://github.com/szmousavi/SLR-of-Security-API-Misuse-Detection
[27] Virginia Braun and Victoria Clarke. 2006. Using thematic analysis in psychology. *Qualitative Research in Psychology* 3, 2 (2006), 77–101.
[28] Pierre Carbonnelle. 2023. PYPL Popularity of Programming Language. (2023). Retrieved October 29, 2023 from https://pypl.github.io/PYPL.html
[29] OpenSSL. (2023). Retrieved June 10, 2023 from https://www.openssl.org/
[30] Nat Sakimura, John Bradley, Mike Jones, Breno de Medeiros, and Chuck Mortimore. 2014. OpenID Connect Core 1.0 Incorporating Errata Set 1. OpenID Foundation. Retrieved from https://openid.net/specs/openid-connect-core-1_0.html

[31] Google. 2015. New in Android Samples: Authenticating to Remote Servers using the Fingerprint API. (2015). Retrieved June 10, 2023 from https://android-developers.googleblog.com/2015/10/new-in-android-samples-authenticating.html
[32] OWASP-MASTG. 2017. Local Authentication on Android. (2017). Retrieved June 10, 2023 from https://github.com/OWASP/owasp-mastg/blob/master/Document/0x05f-Testing-Local-Authentication.md
[33] YubiCo. 2017. YubiKeys. (2017). Retrieved June 10, 2023 from https://www.yubico.com/products/yubikey-hardware/
[34] Spring. Spring Security. ([n. d.]). Retrieved June 10, 2023 from https://spring.io/projects/spring-security
[35] Google. 2020. SafetyNet Attestation API. (2020). Retrieved June 10, 2023 from https://developer.android.com/training/safetynet/attestation
[36] Kevin Allix, Tegawendé F. Bissyandé, Jacques Klein, and Yves Le Traon. 2016. Androzoo: Collecting millions of Android apps for the research community. In *Proceedings of the 13th International Conference on Mining Software Repositories*. 468–471.
[37] Felix Fischer, Huang Xiao, Ching-Yu Kao, Yannick Stachelscheid, Benjamin Johnson, Danial Razar, Paul Fawkesley, Nat Buckley, Konstantin Böttinger, Paul Muntean, et al. 2019. Stack overflow considered helpful! Deep learning security nudges towards stronger cryptography. In *Proceedings of the USENIX Security Symposium*. 339–356.
[38] Elaine Barker, William Burr, Alicia Jones, Timothy Polk, Scott Rose, Miles Smid, Quynh Dang, and others. 2012. Recommendation for key management Part 3: Application-specific key management guidance. *National Institute of Standards*. Retrieved from https://nvlpubs.nist.gov/nistpubs/SpecialPublications/NIST.SP.800-57pt3r1.pdf
[39] Oracle. 2022. JDK 19 Documentation. (2022). Retrieved June 10, 2023 from https://docs.oracle.com/en/java/javase/19/
[40] Elaine Barker and Quynh Dang. 2016. Recommendation for Key Management – Part 1: General (Revision 4). *National Institute of Standards*. Retrieved from https://nvlpubs.nist.gov/nistpubs/SpecialPublications/NIST.SP.800-57pt1r4.pdf
[41] Daniel J. Bernstein, Yun-An Chang, Chen-Mou Cheng, Li-Ping Chou, Nadia Heninger, Tanja Lange, and Nicko Van Someren. 2013. Factoring RSA keys from certified smart cards: Coppersmith in the wild. In *Advances in Cryptology-ASIACRYPT 2013: 19th International Conference on the Theory and Application of Cryptology and Information Security, Bengaluru, India, December 1–5, 2013, Proceedings, Part II 19*. Springer, 341–360.
[42] Hugo Krawczyk. 1992. How to predict congruential generators. *Journal of Algorithms* 13, 4 (1992), 527–545.
[43] Kathleen Moriarty, Burt Kaliski, and Andreas Rusch. 2017. PKCS #5: Password-Based Cryptography Specification Version 2.1. RFC Editor. DOI: https://doi.org/10.17487/RFC8018
[44] Richard Barnes, Martin Thomson, Alfredo Pironti, and Adam Langley. 2015. Deprecating Secure Sockets Layer Version 3.0. RFC Editor. DOI: https://doi.org/10.17487/RFC7568. Accessed May 26, 2025.
[45] K. Moriarty and S. Farrell. 2021. Deprecating TLSv1.0 and TLSv1. (2021). Retrieved June 10, 2023 from https://tools.ietf.org/html/draft-ietf-tls-oldversions-deprecate-12
[46] Sean Turner and Tim Polk. 2011. *Prohibiting Secure Sockets Layer (SSL) Version 2.0*. Technical Report.
[47] Moxie Marlinspike. 2009. More tricks for defeating SSL in practice. Talk presented at Black Hat USA. Retrieved from https://www.blackhat.com/presentations/bh-usa-09/MARLINSPIKE/BHUSA09-Marlinspike-DefeatSSL-SLIDES.pdf
[48] Moxie Marlinspike. 2009. New Tricks for Defeating SSL in Practice. Talk presented at Black Hat DC. Retrieved from https://www.blackhat.com/presentations/bh-dc-09/Marlinspike/BlackHat-DC-09-Marlinspike-Defeating-SSL.pdf
[49] Common Weakness Enumeration. CWE-306: Missing Authentication for Critical Function. ([n. d.]). Retrieved June 10, 2023 from https://cwe.mitre.org/data/definitions/306.html
[50] Tamjid Al Rahat, Yu Feng, and Yuan Tian. 2022. Cerberus: Query-driven scalable vulnerability detection in OAuth service provider implementations. In *Proceedings of the 2022 ACM SIGSAC Conference on Computer and Communications Security*. 2459–2473.
[51] Nat Sakimura, John Bradley, and Naveen Agarwal. 2015. *Proof Key for Code Exchange by OAuth Public Clients*. Technical Report.
[52] Common Weakness Enumeration. CWE-862: Missing Authorization. (2023). Retrieved June 10, 2023 from https://cwe.mitre.org/data/definitions/862.html
[53] Common Weakness Enumeration. CWE-863: Incorrect Authorization. (2023). Retrieved June 10, 2023 from https://cwe.mitre.org/data/definitions/863.html
[54] Marc Stevens, Elie Bursztein, Pierre Karpman, Ange Albertini, Yarik Markov, Alex Petit-Bianco, and Clément Baisse. 2017. Announcing the first SHA-1 collision. Google Security Blog. Retrieved from https://security.googleblog.com/2017/02/announcing-first-sha1-collision.html
[55] Patrick Lam, Eric Bodden, Ondrej Lhoták, and Laurie Hendren. 2011. The Soot framework for Java program analysis: A retrospective. In *Proceedings of the Cetus Users and Compiler Infastructure Workshop*.
[56] Tumbleson Connor and Wiśniewski Ryszard. 2010. Apktool - A Tool for Reverse Engineering Android apk Files. (2010). Retrieved June 10, 2023 from https://ibotpeaches.github.io/Apktool/
[57] Anthony Desnos. 2012. Reverse Engineering and Pentesting for Android Applications. (2012). Retrieved June 10, 2023 from https://github.com/androguard/androguard

[58] IBM: T.J. Watson Libraries for Analysis WALA. (2023). Retrieved June 10, 2023 from https://wala.sourceforge.net/
[59] Harshal Tupsamudre, Monika Sahu, Kumar Vidhani, and Sachin Lodha. 2020. Fixing the fixes: Assessing the solutions of sast tools for securing password storage. In *Financial Cryptography and Data Security: FC 2020 International Workshops, AsiaUSEC, CoDeFi, VOTING, and WTSC, Kota Kinabalu, Malaysia, February 14, 2020, Revised Selected Papers 24*. Springer, 192–206.
[60] Steven Arzt, Siegfried Rasthofer, Christian Fritz, Eric Bodden, Alexandre Bartel, Jacques Klein, Yves Le Traon, Damien Octeau, and Patrick McDaniel. 2014. Flowdroid: Precise context, flow, field, object-sensitive and lifecycle-aware taint analysis for Android apps. *ACM Sigplan Notices* 49, 6 (2014), 259–269.
[61] Baowen Xu, Ju Qian, Xiaofang Zhang, Zhongqiang Wu, and Lin Chen. 2005. A brief survey of program slicing. *ACM SIGSOFT Software Engineering Notes* 30, 2 (2005), 1–36.
[62] James C. King. 1976. Symbolic execution and program testing. *Communications of the ACM* 19, 7 (1976), 385–394.
[63] Alexandre Braga, Ricardo Dahab, Nuno Antunes, Nuno Laranjeiro, and Marco Vieira. 2017. Practical evaluation of static analysis tools for cryptography: Benchmarking method and case study. In *Proceedings of the International Symposium on Software Reliability Engineering*. IEEE, 170–181.
[64] 2019. CryptoAPI-Bench. (2019). Retrieved June 10, 2023 from https://github.com/CryptoGuardOSS/cryptoapi-bench
[65] 2020. ApacheCryptoAPI-Bench. (2020). Retrieved June 10, 2023 from https://github.com/CryptoAPI-Bench/ApacheCryptoAPI-Bench
[66] Sven Amann, Sarah Nadi, Hoan A. Nguyen, Tien N. Nguyen, and Mira Mezini. 2016. MUBench: A benchmark for API-misuse detectors. In *Proceedings of the 13th International Conference on Mining Software Repositories*. 464–467.
[67] 2016. MUBench. (2016). Retrieved June 10, 2023 from https://GitHub.com/stg-tud/MUBench
[68] 2021. OWASP Benchmark. (2021). Retrieved June 10, 2023 from https://owasp.org/www-project-benchmark/
[69] 2015. DroidBench. (2015). Retrieved June 10, 2023 from https://GitHub.com/secure-software-engineering/DroidBench
[70] 2017. ICC-Bench. (2017). Retrieved June 10, 2023 from https://GitHub.com/fgwei/ICC-Bench
[71] Na Meng, Stefan Nagy, Danfeng Yao, Wenjie Zhuang, and Gustavo Arango Argoty. 2018. Secure coding practices in Java: Challenges and vulnerabilities. In *Proceedings of the 40th International Conference on Software Engineering*. 372–383.
[72] Amiangshu Bosu, Fang Liu, Danfeng Yao, and Gang Wang. 2017. Collusive data leak and more: Large-scale threat analysis of inter-app communications. In *Proceedings of the 2017 ACM on Asia Conference on Computer and Communications Security*. 71–85.
[73] Yuhong Nan, Zhemin Yang, Xiaofeng Wang, Yuan Zhang, Donglai Zhu, and Min Yang. 2018. Finding clues for your secrets: Semantics-driven, learning-based privacy discovery in mobile apps. In *Proceedings of the 2018 Network and Distributed System Security Symposium (NDSS)*. Internet Society, San Diego, CA, USA. Retrieved from https://yangzhemin.github.io/papers/cluefinder-ndss2018.pdf
[74] Chaoshun Zuo, Zhiqiang Lin, and Yinqian Zhang. 2019. Why does your data leak? Uncovering the data leakage in cloud from mobile apps. In *Proceedings of the 2019 IEEE Symposium on Security and Privacy*. IEEE, 1296–1310.
[75] Douglas Kunda and Mumbi Chishimba. 2021. A survey of Android mobile phone authentication schemes. *Mobile Networks and Applications* 26, 6 (2021), 2558–2566.
[76] Chengcheng Wan, Shicheng Liu, Henry Hoffmann, Michael Maire, and Shan Lu. 2021. Are machine learning cloud APIs used correctly?. In *Proceedings of the 2021 IEEE/ACM 43rd International Conference on Software Engineering* . IEEE, 125–137.
[77] Chengcheng Wan, Shicheng Liu, Sophie Xie, Yifan Liu, Henry Hoffmann, Michael Maire, and Shan Lu. 2022. Automated testing of software that uses machine learning apis. In *Proceedings of the 44th International Conference on Software Engineering*. 212–224.
[78] Moshi Wei, Nima Shiri Harzevili, YueKai Huang, Jinqiu Yang, Junjie Wang, and Song Wang. 2024. Demystifying and detecting misuses of deep learning APIs. In *Proceedings of the IEEE/ACM 46th International Conference on Software Engineering*. 1–12.
[79] Xinyi Hou, Yanjie Zhao, Yue Liu, Zhou Yang, Kailong Wang, Li Li, Xiapu Luo, David Lo, John Grundy, and Haoyu Wang. 2024. Large language models for software engineering: A systematic literature review. *ACM Transactions on Software Engineering and Methodology* 33, 8 (2024), 1–79.
[80] Steven I. Ross, Fernando Martinez, Stephanie Houde, Michael Muller, and Justin D. Weisz. 2023. The programmer's assistant: Conversational interaction with a large language model for software development. In *Proceedings of the 28th International Conference on Intelligent User Interfaces*. 491–514.
[81] Daye Nam, Andrew Macvean, Vincent Hellendoorn, Bogdan Vasilescu, and Brad Myers. 2024. Using an LLM to help with code understanding. In *Proceedings of the IEEE/ACM 46th International Conference on Software Engineering*. 1–13.
[82] Chunqiu Steven Xia, Yuxiang Wei, and Lingming Zhang. 2023. Automated program repair in the era of large pre-trained language models. In *Proceedings of the 2023 IEEE/ACM 45th International Conference on Software Engineering* . IEEE, 1482–1494.

[83] Zahra Mousavi, Chadni Islam, Kristen Moore, Alsharif Abuadbba, and M. Ali Babar. 2024. An investigation into misuse of Java security APIs by large language models. In *Proceedings of the 19th ACM Asia Conference on Computer and Communications Security*. 1299–1315.
[84] Google. Biometrics API. (2023). Retrieved October 29, 2023 from https://developer.android.com/reference/android/hardware/biometrics/package-summary
[85] Google. Play Integrity. (2023). Retrieved October 29, 2023 https://developer.android.com/google/play/integrity
[86] Hao Zhong and Na Meng. 2024. Compiler-directed migrating API callsite of client code. In *Proceedings of the IEEE/ACM 46th International Conference on Software Engineering*. 1–12.
[87] Feargus Pendlebury, Fabio Pierazzi, Roberto Jordaney, Johannes Kinder, and Lorenzo Cavallaro. 2019. {TESSERACT}: Eliminating experimental bias in malware classification across space and time. In *Proceedings of the 28th USENIX Security Symposium*. 729–746.
[88] Tiago Espinha Gasiba, Ulrike Lechner, and Maria Pinto-Albuquerque. 2020. Sifu-a cybersecurity awareness platform with challenge assessment and intelligent coach. *Cybersecurity* 3, 1 (2020), 24.
[89] Tiago Gasiba, Ulrike Lechner, Maria Pinto-Albuquerque, and Alae Zouitni. 2020. Design of secure coding challenges for cybersecurity education in the industry. In *Proceedings of the International Conference on the Quality of Information and Communications Technology*. Springer, 223–237.
[90] Tiago Gasiba, Ulrike Lechner, and Maria Pinto-Albuquerque. 2021. Cybersecurity challenges for software developer awareness training in industrial environments. In *Proceedings of the Innovation Through Information Systems: Volume II: A Collection of Latest Research on Technology Issues*. Springer, 370–387.
[91] Georgios Lampropoulos and Kinshuk. 2024. Virtual reality and gamification in education: A systematic review. *Educational Technology Research and Development* 72, 3 (2024), 1691–1785.

Received 14 June 2023; revised 1 May 2025; accepted 8 May 2025